



DD2350 ALGORITMER, DATASTRUKTURER OCH KOMPLEXITET

# Övning 5

## Grafalgoritmer och undre gränser

Marcus | HT 2026

Uppgifter och lösningssidéer: Viggo Kann (2026-08-25). Lånade idéer ur tidigare övningsbilder: Anton Grensjö (2017), Lisa Li (2019) och Alice.

Sandfärgade bilder med kanin 🐰 är kaninhål: fördjupningar som går att hoppa över.

# Dagens program

- |                            |                    |
|----------------------------|--------------------|
| 1. Spännande träd          | Prim, Kruskal      |
| 2. Förändrat flöde         | $\pm 1$ på en kant |
| 3. Eulercykel              | linjär tid         |
| 4. Julklappsfördelning     | matchning          |
| 5. Undre gräns för sökning | beslutsträd        |
| 6. Laga julgransbelysning  | motståndare        |

## Tre sorters bilder

-  Lösningar och exempel
-   Kaninhål (valfria)
-   Vanliga missuppfattningar
-  **Kontrast** Variationsmönster

Övningen gräver ner sig i alla kaninhål. Kaninhålen ligger på egna bilder, så den som har bråttom kan hoppa över dem. Den som inte har bråttom kan också det.

## Om upplägget: variationsteori

**Idé (Marton):** man urskiljer en aspekt av något först när man upplever att den *varierar* medan annat hålls fast. Den som bara sett röda bollar vet inte att bollar har färg.

- ▶ **Lärandeobjekt:** det vi vill att ni ska kunna.
- ▶ **Kritiska aspekter:** det man måste urskilja för att kunna det.
- ▶ Varje uppgift börjar med en bild som säger vilka aspekter vi varierar.

### Kontrast

Visa vad det *inte* är (Prim vs Dijkstra).

### Separation

Variera en aspekt, håll resten fast.

### Generalisering

Håll aspekten fast, variera resten.

### Fusion

Variera flera aspekter samtidigt.

**Exempel** Fast: grafen · Varierar: algoritmen (Prim/Dijkstra) · Urskiljs: att "girig från en rot" inte säger vad nyckeln är

## 1

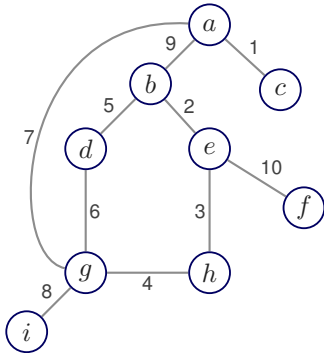
## Spännande träd

**Lärandeobjekt:** köra Prim och Kruskal för hand — och se *varför* två olika giriga val båda blir rätt.

**Kritiska aspekter** (det vi kommer att variera):

- ▶ Kandidatmängden: *hela grafen* (Kruskal) eller *kanten ut ur trädet* (Prim).
- ▶ Tillståndet under körningen: skog eller träd.
- ▶ Vad MST beror på: bara vikternas *ordning* (kontrast: kortaste väg).

# Uppgift 1: Spännande träd



Visa hur ett minimalt spännande träd hittas i grafen med både **Prims** och **Kruskals** algoritmer.

- **Spännande träd:** delgraf som är ett träd och når *alla* hörn. Alltså exakt  $|V| - 1$  kanter; här  $9 - 1 = 8$ .
- **Minimalt:** minsta möjliga summa av kantvikterna.
- Båda algoritmerna är *giriga*. De är bara giriga på olika saker.

## Två giriga algoritmer — och en minnesregel

### Kruskal

Ta den **globalt** billigaste kanten, i hela grafen, som förbinder två *olika komponenter*. Tillståndet är en *skog*.

### Prim

Ta den billigaste kanten som går **ut ur trädet**. Tillståndet är ett enda *träd* som växer från en rot.

Krusk**AL** är Glob**AL**

Prim är lokal. (Rimmar inte. Det är därför ingen kommer ihåg det.)

**Kontrast** Fast: grafen, girigheten · **Varierar**: kandidatmängden · **Urskiljs**: global kantlista mot lokal snittkant

# Pseudokod

## Kruskal( $G, w$ )

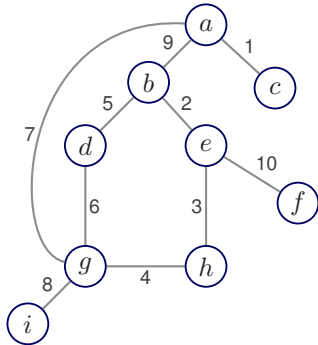
- 1:  $T \leftarrow \emptyset$
- 2: **för alla**  $v \in V$  **gör**
- 3:      $\text{MAKESET}(v)$
- 4: sortera  $E$  i växande vikt
- 5: **för alla**  $(u, v) \in E$  i sorterad ordning **gör**
- 6:     **om**  $\text{FIND}(u) \neq \text{FIND}(v)$  **så**
- 7:          $T \leftarrow T \cup \{(u, v)\}$
- 8:      $\text{UNION}(u, v)$
- 9: **returnera**  $T$

## Prim( $G, w, r$ )

- 1: **för alla**  $v \in V$  **gör**
- 2:      $\text{key}[v] \leftarrow \infty; \pi[v] \leftarrow \text{NIL}$
- 3:  $\text{key}[r] \leftarrow 0$
- 4:  $Q \leftarrow V$  ▷ prioritet  $\text{key}$
- 5: **medan**  $Q \neq \emptyset$  **gör**
- 6:      $u \leftarrow \text{EXTRACTMIN}(Q)$
- 7:     **för alla**  $(u, v) \in E$  **gör**
- 8:         **om**  $v \in Q$  **och**  $w(u, v) < \text{key}[v]$  **så**
- 9:              $\pi[v] \leftarrow u; \text{key}[v] \leftarrow w(u, v)$
- 10: **returnera**  $\{(\pi[v], v) : v \neq r\}$

**⚠ Missuppfattning:** "Villkoret  $v \in Q$  är en detalj." Utan det kan ett redan utplockat hörn få ny nyckel och ny förälder — och då är resultatet inte längre ett träd. (Just den buggen finns i äldre övningsbilder.)

# Kruskal på uppgiftens graf



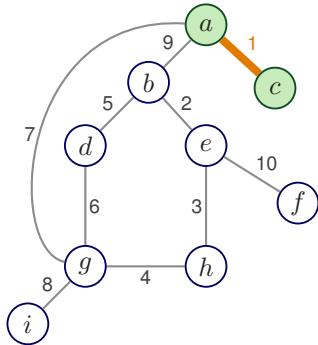
## Alla kanter, globalt sorterade

- 1  $a-c$
- 2  $b-e$
- 3  $e-h$
- 4  $g-h$
- 5  $b-d$
- 6  $d-g$
- 7  $a-g$
- 8  $g-i$
- 9  $a-b$
- 10  $e-f$

Komponenter: 9   Kanter i  $T$ : 0   Vikt: 0

Sortera *alla* kanter. Varje hörn är en egen komponent.

# Kruskal på uppgiftens graf



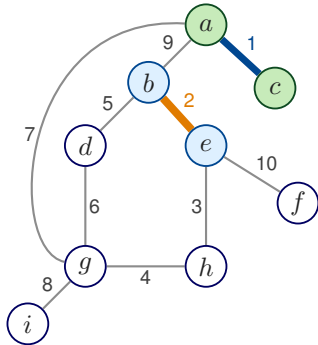
## Alla kanter, globalt sorterade

|   |    |       |   |
|---|----|-------|---|
| ▶ | 1  | $a-c$ | ✓ |
|   | 2  | $b-e$ |   |
|   | 3  | $e-h$ |   |
|   | 4  | $g-h$ |   |
|   | 5  | $b-d$ |   |
|   | 6  | $d-g$ |   |
|   | 7  | $a-g$ |   |
|   | 8  | $g-i$ |   |
|   | 9  | $a-b$ |   |
|   | 10 | $e-f$ |   |

Komponenter: 8   Kanter i  $T$ : 1   Vikt: 1

$a-c$  förbinder två olika komponenter  $\Rightarrow$  ta den.

# Kruskal på uppgiftens graf



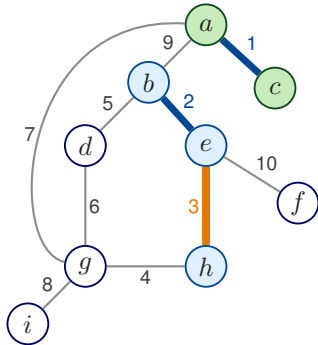
## Alla kanter, globalt sorterade

|    |       |   |
|----|-------|---|
| 1  | $a-c$ | ✓ |
| 2  | $b-e$ | ✓ |
| 3  | $e-h$ |   |
| 4  | $g-h$ |   |
| 5  | $b-d$ |   |
| 6  | $d-g$ |   |
| 7  | $a-g$ |   |
| 8  | $g-i$ |   |
| 9  | $a-b$ |   |
| 10 | $e-f$ |   |

Komponenter: 7   Kanter i  $T$ : 2   Vikt: 3

$b-e$ : på andra sidan grafen. Kruskal bryr sig inte om *var* kanten är — bara att den är billigast *globalt*.

# Kruskal på uppgiftens graf



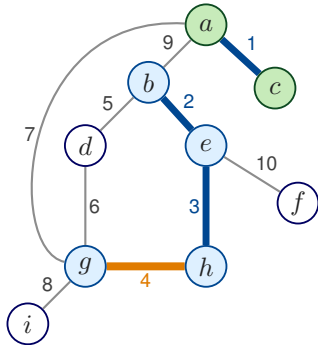
## Alla kanter, globalt sorterade

- 1  $a-c$  ✓
- 2  $b-e$  ✓
- ▶ 3  $e-h$  ✓
- 4  $g-h$
- 5  $b-d$
- 6  $d-g$
- 7  $a-g$
- 8  $g-i$
- 9  $a-b$
- 10  $e-f$

Komponenter: 6   Kanter i  $T$ : 3   Vikt: 6

$e-h \Rightarrow$  ta den.

# Kruskal på uppgiftens graf



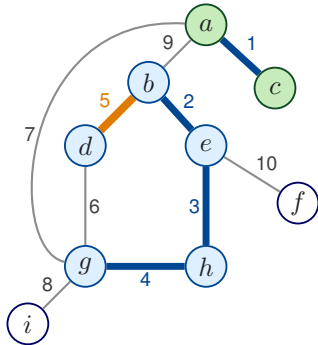
## Alla kanter, globalt sorterade

|     |       |   |
|-----|-------|---|
| 1   | $a-c$ | ✓ |
| 2   | $b-e$ | ✓ |
| 3   | $e-h$ | ✓ |
| ▶ 4 | $g-h$ | ✓ |
| 5   | $b-d$ |   |
| 6   | $d-g$ |   |
| 7   | $a-g$ |   |
| 8   | $g-i$ |   |
| 9   | $a-b$ |   |
| 10  | $e-f$ |   |

Komponenter: 5   Kanter i  $T$ : 4   Vikt: 10

$g-h \Rightarrow$  ta den.

# Kruskal på uppgiftens graf



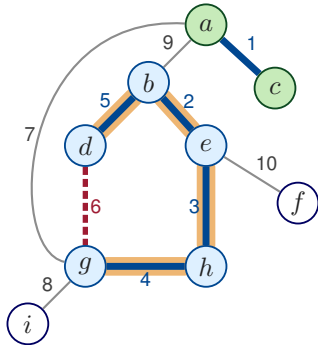
## Alla kanter, globalt sorterade

|     |       |   |
|-----|-------|---|
| 1   | $a-c$ | ✓ |
| 2   | $b-e$ | ✓ |
| 3   | $e-h$ | ✓ |
| 4   | $g-h$ | ✓ |
| ▶ 5 | $b-d$ | ✓ |
| 6   | $d-g$ |   |
| 7   | $a-g$ |   |
| 8   | $g-i$ |   |
| 9   | $a-b$ |   |
| 10  | $e-f$ |   |

Komponenter: 4   Kanter i  $T$ : 5   Vikt: 15

$b-d \Rightarrow$  ta den.

# Kruskal på uppgiftens graf



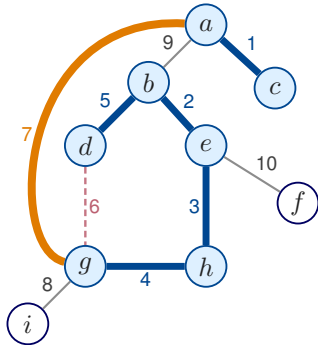
## Alla kanter, globalt sorterade

|     |       |                                       |
|-----|-------|---------------------------------------|
| 1   | $a-c$ | ✓                                     |
| 2   | $b-e$ | ✓                                     |
| 3   | $e-h$ | ✓                                     |
| 4   | $g-h$ | ✓                                     |
| 5   | $b-d$ | ✓                                     |
| ▶ 6 | $d-g$ | ✗ samma komponent $\Rightarrow$ cykel |
| 7   | $a-g$ |                                       |
| 8   | $g-i$ |                                       |
| 9   | $a-b$ |                                       |
| 10  | $e-f$ |                                       |

Komponenter: 4   Kanter i  $T$ : 5   Vikt: 15

$d-g$ :  $d$  och  $g$  hänger redan ihop via  $d-b-e-h-g$ . Kanten skulle sluta en cykel.

# Kruskal på uppgiftens graf



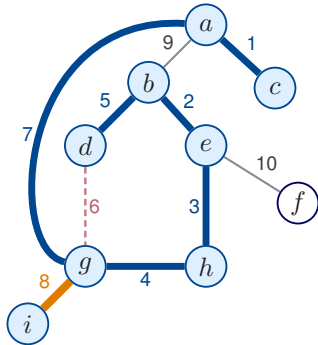
## Alla kanter, globalt sorterade

|    |       |                                       |
|----|-------|---------------------------------------|
| 1  | $a-c$ | ✓                                     |
| 2  | $b-e$ | ✓                                     |
| 3  | $e-h$ | ✓                                     |
| 4  | $g-h$ | ✓                                     |
| 5  | $b-d$ | ✓                                     |
| 6  | $d-g$ | ✗ samma komponent $\Rightarrow$ cykel |
| 7  | $a-g$ | ✓                                     |
| 8  | $g-i$ |                                       |
| 9  | $a-b$ |                                       |
| 10 | $e-f$ |                                       |

Komponenter: 3   Kanter i  $T$ : 6   Vikt: 22

$a-g$  förenar  $\{a, c\}$  med den stora komponenten.

# Kruskal på uppgiftens graf



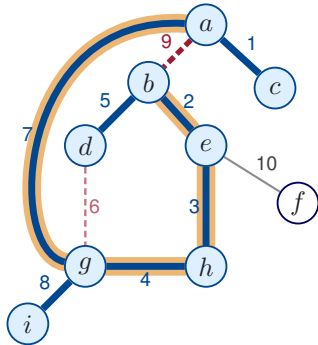
## Alla kanter, globalt sorterade

|     |       |                                       |
|-----|-------|---------------------------------------|
| 1   | $a-c$ | ✓                                     |
| 2   | $b-e$ | ✓                                     |
| 3   | $e-h$ | ✓                                     |
| 4   | $g-h$ | ✓                                     |
| 5   | $b-d$ | ✓                                     |
| 6   | $d-g$ | ✗ samma komponent $\Rightarrow$ cykel |
| 7   | $a-g$ | ✓                                     |
| ▶ 8 | $g-i$ | ✓                                     |
| 9   | $a-b$ |                                       |
| 10  | $e-f$ |                                       |

Komponenter: 2   Kanter i  $T$ : 7   Vikt: 30

$g-i \Rightarrow$  ta den.

# Kruskal på uppgiftens graf



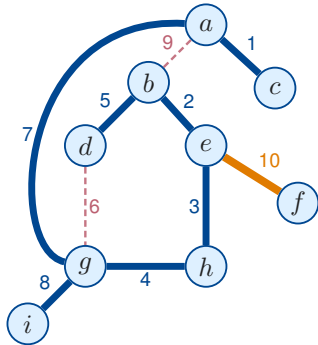
## Alla kanter, globalt sorterade

|    |       |                                       |
|----|-------|---------------------------------------|
| 1  | $a-c$ | ✓                                     |
| 2  | $b-e$ | ✓                                     |
| 3  | $e-h$ | ✓                                     |
| 4  | $g-h$ | ✓                                     |
| 5  | $b-d$ | ✓                                     |
| 6  | $d-g$ | ✗ samma komponent $\Rightarrow$ cykel |
| 7  | $a-g$ | ✓                                     |
| 8  | $g-i$ | ✓                                     |
| 9  | $a-b$ | ✗ samma komponent $\Rightarrow$ cykel |
| 10 | $e-f$ |                                       |

Komponenter: 2   Kanter i  $T$ : 7   Vikt: 30

$a-b$ : redan förbundna via  $a-g-h-e-b \Rightarrow$  cykel.

# Kruskal på uppgiftens graf



## Alla kanter, globalt sorterade

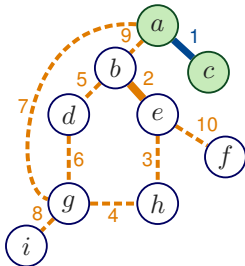
|      |       |                                       |
|------|-------|---------------------------------------|
| 1    | $a-c$ | ✓                                     |
| 2    | $b-e$ | ✓                                     |
| 3    | $e-h$ | ✓                                     |
| 4    | $g-h$ | ✓                                     |
| 5    | $b-d$ | ✓                                     |
| 6    | $d-g$ | ✗ samma komponent $\Rightarrow$ cykel |
| 7    | $a-g$ | ✓                                     |
| 8    | $g-i$ | ✓                                     |
| 9    | $a-b$ | ✗ samma komponent $\Rightarrow$ cykel |
| ▶ 10 | $e-f$ | ✓                                     |

Komponenter: 1   Kanter i  $T$ : 8   Vikt: 40

$e-f \Rightarrow$  ta den.  $|V| - 1 = 8$  kanter, vikt 40. Klart.

# Samma tillstånd, olika kandidater

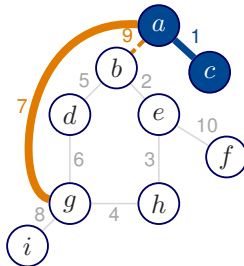
Kruskal efter första kanten



Kandidater: *alla* kanter mellan olika komponenter — här alla som återstår.

Väljer **2** (b–e) — långt bort. Krusk**AL**: glob**AL**.

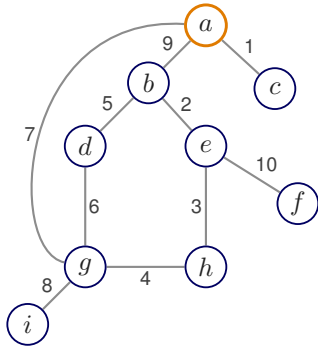
Prim efter första kanten



Kandidater: bara kanter *ut ur träd*et {a, c}: 7 och 9.  
Väljer **7** (a–g), trots att 2 finns. Prim: lokal.

**Kontrast** Fast: grafen och första steget · **Varierar**: algoritmen · **Urskiljs**: vilken mängd "billigast" gäller

## Prim på uppgiftens graf (rot $a$ )



**Prioritetskö  $Q$  (nyckel = billigaste kant in i trädet)**

| hörn | key | $\pi$ |
|------|-----|-------|
|------|-----|-------|

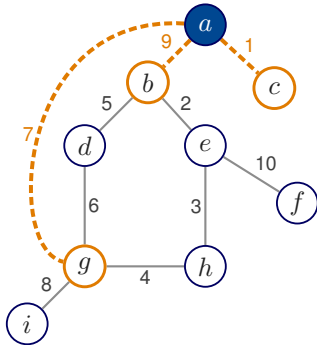
|     |   |   |
|-----|---|---|
| $a$ | 0 | – |
|-----|---|---|

+ 8 hörn med nyckel  $\infty$

Utplockade: –

Starta i  $a$ :  $key[a] = 0$ , alla andra  $\infty$ . Alla hörn ligger i kön.

## Prim på uppgiftens graf (rot $a$ )



**Prioritetskö  $Q$  (nyckel = billigaste kant in i trädet)**

| hörn | key | $\pi$ |
|------|-----|-------|
|------|-----|-------|

|     |   |          |
|-----|---|----------|
| $c$ | 1 | $a$ (ny) |
|-----|---|----------|

|     |   |          |
|-----|---|----------|
| $g$ | 7 | $a$ (ny) |
|-----|---|----------|

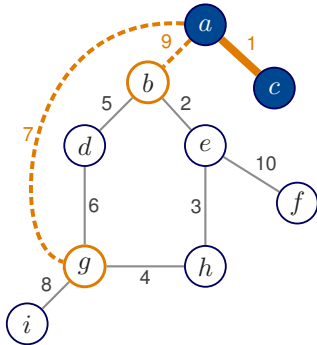
|     |   |          |
|-----|---|----------|
| $b$ | 9 | $a$ (ny) |
|-----|---|----------|

|                              |  |  |
|------------------------------|--|--|
| + 5 hörn med nyckel $\infty$ |  |  |
|------------------------------|--|--|

Utplockade:  $a$

Plocka ut  $a$ . Grannarna får nycklar:  $c$  (1),  $g$  (7),  $b$  (9).

## Prim på uppgiftens graf (rot $a$ )



**Prioritetskö  $Q$  (nyckel = billigaste kant in i trädet)**

| hörn | key | $\pi$ |
|------|-----|-------|
|------|-----|-------|

|     |   |     |
|-----|---|-----|
| $g$ | 7 | $a$ |
|-----|---|-----|

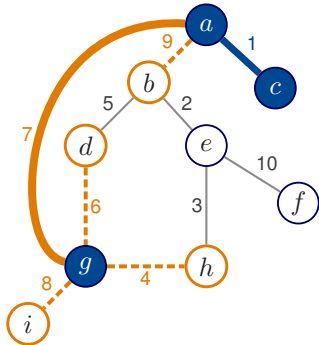
|     |   |     |
|-----|---|-----|
| $b$ | 9 | $a$ |
|-----|---|-----|

+ 5 hörn med nyckel  $\infty$

Utplockade:  $a, c$

Plocka ut  $c$  (1). Inga nya grannar. Kön är lokal: bara kanter *ut ur trädet* räknas.

## Prim på uppgiftens graf (rot $a$ )



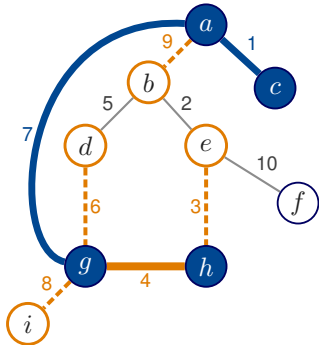
**Prioritetskö  $Q$  (nyckel = billigaste kant in i trädet)**

| hörn                         | key | $\pi$    |
|------------------------------|-----|----------|
| $h$                          | 4   | $g$ (ny) |
| $d$                          | 6   | $g$ (ny) |
| $i$                          | 8   | $g$ (ny) |
| $b$                          | 9   | $a$      |
| + 2 hörn med nyckel $\infty$ |     |          |

Utplockade:  $a, c, g$

Plocka ut  $g$  (7). Nya nycklar:  $h$  (4),  $d$  (6),  $i$  (8).

## Prim på uppgiftens graf (rot $a$ )



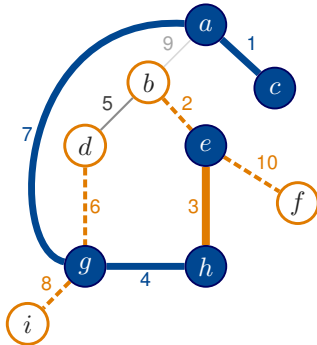
**Prioritetskö  $Q$  (nyckel = billigaste kant in i trädet)**

| hörn                         | key | $\pi$    |
|------------------------------|-----|----------|
| $e$                          | 3   | $h$ (ny) |
| $d$                          | 6   | $g$      |
| $i$                          | 8   | $g$      |
| $b$                          | 9   | $a$      |
| + 1 hörn med nyckel $\infty$ |     |          |

Utplockade:  $a, c, g, h$

Plocka ut  $h$  (4). Ny nyckel:  $e$  (3).

## Prim på uppgiftens graf (rot $a$ )



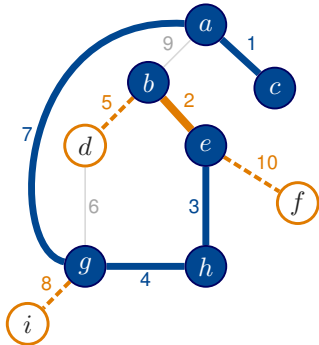
Prioritetskö  $Q$  (nyckel = billigaste kant in i trädet)

| hörn | key | $\pi$       |
|------|-----|-------------|
| $b$  | 2   | $e$ (var 9) |
| $d$  | 6   | $g$         |
| $i$  | 8   | $g$         |
| $f$  | 10  | $e$ (ny)    |

Utplockade:  $a, c, g, h, e$

Plocka ut  $e$  (3). DECREASE-KEY:  $b$  går från 9 till 2. Ny:  $f$  (10).

## Prim på uppgiftens graf (rot $a$ )



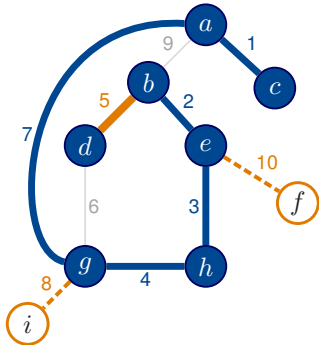
**Prioritetskö  $Q$  (nyckel = billigaste kant in i trädet)**

| hörn | key | $\pi$       |
|------|-----|-------------|
| $d$  | 5   | $b$ (var 6) |
| $i$  | 8   | $g$         |
| $f$  | 10  | $e$         |

Utplockade:  $a, c, g, h, e, b$

Plocka ut  $b$  (2). DECREASE-KEY:  $d$  går från 6 till 5.

## Prim på uppgiftens graf (rot $a$ )



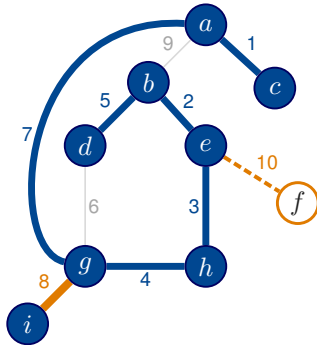
**Prioritetskö  $Q$  (nyckel = billigaste kant in i trädet)**

| hörn | key | $\pi$ |
|------|-----|-------|
| $i$  | 8   | $g$   |
| $f$  | 10  | $e$   |

Utplockade:  $a, c, g, h, e, b, d$

Plocka ut  $d$  (5). Kanten  $d-g$  (6) blev aldrig trädkant.

# Prim på uppgiftens graf (rot $a$ )



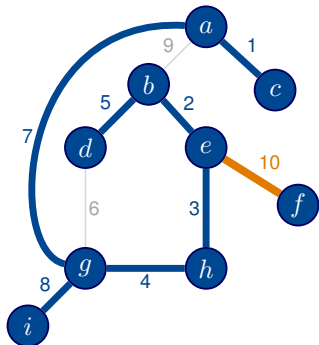
**Prioritetskö  $Q$  (nyckel = billigaste kant in i trädet)**

| hörn | key | $\pi$ |
|------|-----|-------|
| $f$  | 10  | $e$   |

Utplockade:  $a, c, g, h, e, b, d, i$

Plocka ut  $i$  (8).

## Prim på uppgiftens graf (rot $a$ )



**Prioritetskö  $Q$  (nyckel = billigaste kant in i trädet)**

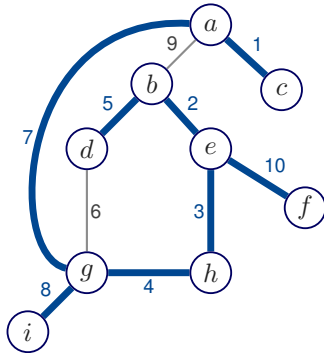
| hörn | key | $\pi$ |
|------|-----|-------|
|------|-----|-------|

|       |  |  |
|-------|--|--|
| (tom) |  |  |
|-------|--|--|

Utplockade:  $a, c, g, h, e, b, d, i, f$

Plocka ut  $f$  (10). Vikt  $1 + 7 + 4 + 3 + 2 + 5 + 8 + 10 = 40$ . Samma träd som Kruskal (unika vikter!).

# Facit och kontroll

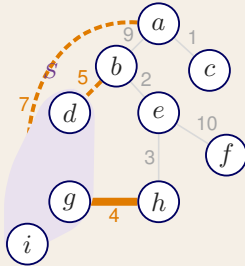


- ▶ Kanter: 1, 2, 3, 4, 5, 7, 8, 10. Vikt: 40.
- ▶ Kruskal förkastar **6** och **9**: båda sluter cykler.
- ▶ Prim från *a* plockar i ordningen *a, c, g, h, e, b, d, i, f*.
- ▶ Samma träd, olika *ordning*. Det är ingen slump: med unika vikter finns bara ett MST.

**Kontroll som alltid fungerar:** rätt antal kanter ( $|V| - 1$ ), ingen cykel, och varje förkastad kant är den tyngsta på sin cykel.

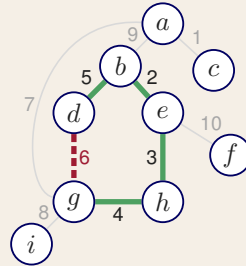
# Kaninhål: Varför fungerar girigheten? Snitt och cykler

Snittegenskapen



Kanterna över snittet  $S = \{d, g, i\}$ : 4, 5, 7. Den lättaste, **4**, ingår i *varje* MST.

Cykelegenskapen

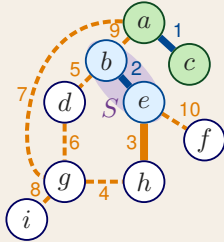


På cykeln  $b-d-g-h-e$  är **6** tyngst. Den ingår i *ingen* MST.

Båda algoritmerna tar i varje steg en kant som är lättast över *något* snitt som ingen vald kant korsar. Därför är båda korrekta. Men snittet spelar olika roll: för Prim *är* det valregeln, för Kruskal finns det bara i beviset. Nästa bild.

# Kaninhål: Snittet: valregel för Prim, bara bevis för Kruskal

## Kruskal efter två kanter



streckat: kandidater  $S$ : bara i beviset

**Valet är globalt.** Kruskal jämför *alla* kanter mellan olika komponenter, 3 till 10 — också 4, 6, 7 och 8, som inte rör  $b$  eller  $e$ . Den tar 3.

**Snittet kommer i efterhand.** Beviset väljer  $S$  = ena ändens komponent, här  $\{b, e\}$ . Kanterna över  $(S, V \setminus S)$  är 3, 5, 9 och 10. Alla går mellan olika komponenter, så alla var kandidater. Alltså är 3 lättast även över snittet.

Globalt lättast  $\Rightarrow$  lättast över snittet. Inte tvärtom.

|         | Vilken kant tas?                                | Snittet                              |
|---------|-------------------------------------------------|--------------------------------------|
| Prim    | lättast ut ur trädets                           | (trädets, resten): är valregeln      |
| Kruskal | lättast av alla kanter mellan olika komponenter | ena ändens komponent: bara i beviset |
| Borůvka | varje komponent tar sin lättaste kant ut        | alla komponenters snitt, samtidigt   |

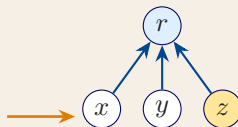
## Kaninhål: Union-Find: hur Kruskal vet vad som hänger ihop

- ▶ Varje komponent är ett träd av föräldrapekare; roten är komponentens namn.
- ▶  $\text{FIND}(x)$ : följ pekarna till roten.
- ▶ UNION: häng den *mindre* roten under den större.
- ▶ **Stigkomprimering**: låt alla på vägen peka direkt på roten.
- ▶ Båda knepen: amorterat  $O(\alpha(n))$  per operation.  $\alpha$  är i praktiken  $\leq 4$ .

före  $\text{FIND}(z)$



efter



**Kruskals tid:** sortering  $O(|E| \log |E|) + |E|$  FIND och  $|V| - 1$  UNION. Sorteringen dominerar. Och  $\log |E| \leq \log |V|^2 = 2 \log |V|$ , så  $O(|E| \log |E|) = O(|E| \log |V|)$ .

## Kaninhål: Prims tid beror på prioritetskön

| Kö                    | EXTRACTMIN        | DECREASEKEY              | Totalt                            |
|-----------------------|-------------------|--------------------------|-----------------------------------|
| osorterad array       | $O( V )$          | $O(1)$                   | $O( V ^2)$                        |
| binär heap            | $O(\log  V )$     | $O(\log  V )$            | $O( E  \log  V )$                 |
| Fibonacciheap         | $O(\log  V )$ am. | $O(1)$ am.               | $O( E  +  V  \log  V )$           |
| "lat" heap med kanter | $O(\log  E )$     | — (dubbletter i stället) | $O( E  \log  V )$ , se nästa bild |

**Tät graf**,  $|E| = \Theta(|V|^2)$ : arrayen vinner.  $O(|V|^2)$  är dessutom *linjärt i grannmatrisens storlek*.

**Gles graf**,  $|E| = O(|V|)$ : heapen vinner,  $O(|V| \log |V|)$ . Samma algoritm, olika datastruktur, olika "bästa".

**Separation** **Fast**: algoritmen (Prim) · **Varierar**: datastrukturen och tätheten · **Urskiljs**: att komplexitet hör till algoritm *plus* datastruktur

## Kaninhål: Lat heap: blir dubbletterna ett problem?

### PrimLat( $G, w, r$ )

```

1:  $T \leftarrow \{r\}; H \leftarrow$  alla kanter  $(r, x)$ 
2: medan  $H \neq \emptyset$  gör
3:    $(u, v) \leftarrow \text{EXTRACTMIN}(H)$ 
4:   om  $v \in T$  så
5:     släng den ▷ inaktuell dubblett
6:   annars
7:      $T \leftarrow T \cup \{v\};$  ta med  $(u, v)$ 
8:   för alla  $(v, x) \in E$  med  $x \notin T$  gör
9:     INSERT( $H, (v, x)$ )

```

Totalt  $O(|E| \log |V|)$ , samma som med DECREASEKEY. Dubbletterna kostar minne,  $O(|E|)$  i stället för  $O(|V|)$ , och en konstant faktor. Inte asymptotisk tid.

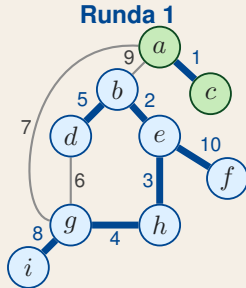
**Ja**, i en fullständig graf ligger ett hörn i heapen en gång för varje granne som hann in i trädet före det: upp till  $|V| - 1$  gånger.

**Men** varje kant läggs in högst en gång, när dess första ände kommer med i trädet. Heapen har aldrig fler än  $|E|$  element.

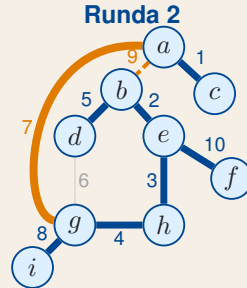
Alltså högst  $|E|$  INSERT och  $|E|$  EXTRACTMIN, var och en  $O(\log |E|)$ . Och  $\log |E| \leq \log |V|^2 = 2 \log |V|$ .

**Olyckliga vikter** behövs i varianten som bara lägger in när nyckeln blir bättre:  $w(i, j) = n(n - i) + j$  för  $i < j$ . Prim från 1 tar hörnen i ordning  $1, 2, \dots, n$ , och varje nytt hörn förbättrar alla återstående nycklar. Också då:  $|E|$  element.

## Kaninhål: Borůvka (1926): alla komponenter väljer samtidigt



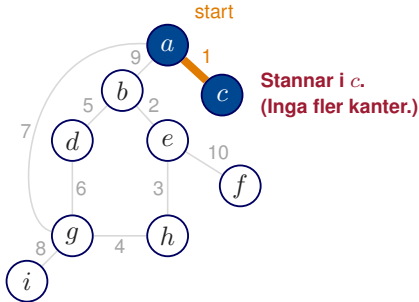
Varje hörn tar sin billigaste kant:  
1, 2, 3, 4, 5, 8, 10. Två komponenter kvar.



Billigaste kanten mellan dem: 7 (inte 9). Klart.

Antalet komponenter minst halveras per runda  $\Rightarrow O(\log |V|)$  rundor  $\rightarrow O(|E|)$ :  $O(|E| \log |V|)$ . Äldst av de tre, och lättast att parallellisera. (Kräver unika vikter eller konsekvent tie-break — annars kan två komponenter välja varsin kant och bilda en cykel.)

## Vanligt fel: "billigaste kanten från där jag står"



**! Missuppfattning:** "Prim = gå längs billigaste kanten från det senaste hörnet."

Från  $a$ : billigast är 1 till  $c$ . Kommer till  $c$ . Stannar i  $c$ .

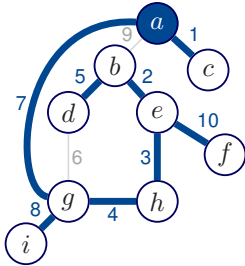
Det där är en *promenad*, inte Prim. Prim tittar på alla kanter ut ur *hela trädet*  $\{a, c\}$ , inte bara ut ur  $c$ .

Och även när promenaden inte kör fast blir den en stig — ett spännande träd behöver kunna förgrena sig.

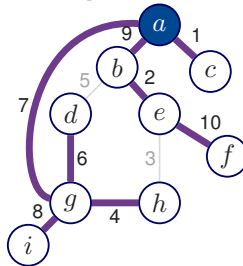
**Kontrast Fast:** grafen, startpunkten · **Varierar:** "ut ur senaste hörnet" / "ut ur trädet" · **Urskiljs:** snittet går runt hela trädet

# Prim är inte Dijkstra

**MST (Prim), vikt 40**



**Kortaste-vägträd från a, vikt 47**



avstånd:  $b$  9,  $c$  1,  $d$  13,  $e$  11,  $f$  21,  $g$  7,  $h$  11,  $i$  15

Samma skelett: kö, EXTRACTMIN, DECREASEKEY. En rad skiljer:

Prim:  $key[v] \leftarrow w(u, v)$

Dijkstra:  $d[v] \leftarrow d[u] + w(u, v)$

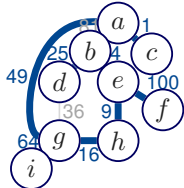
I MST är vägen  $a \rightsquigarrow b$  lång:  $7 + 4 + 3 + 2 = 16$ . Dijkstra tar kanten 9 direkt. MST minimerar *summan över trädet*, inte avståndet från  $a$ .

**Kontrast** Fast: graf, rot, köstruktur · **Varierar:** nyckeln: kantvikt eller avstånd · **Urskiljs:** vad som minimeras

# Ändra vikterna: MST och kortaste-vägträd reagerar olika

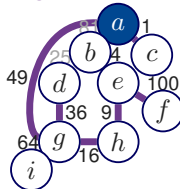
| Ändring                                    | MST           | Kortaste-vägträd från $a$ (förra bildens träd)           |
|--------------------------------------------|---------------|----------------------------------------------------------|
| $w \mapsto w^2$                            | samma         | <b>ändras:</b> $b$ nås nu via $e$ ( $78 < 81$ )          |
| $w \mapsto w - 5$                          | samma         | <b>odefinierat:</b> fram och tillbaka på en negativ kant |
| $w \mapsto w + c$                          | samma         | kan ändras: straffar stigar med många kanter             |
| byt $5 \leftrightarrow 6$ på $b-d$ , $d-g$ | <b>ändras</b> | samma (här)                                              |

MST med vikterna  $w^2$



Samma kanter som förut.

Kortaste-vägträd från  $a$ , vikterna  $w^2$



Inte ett MST. Nytt:  $b$  hänger på  $e$ .

Strikt växande funktion av vikterna  
 $\Rightarrow$  samma MST: Kruskal ser bara ordningen.

Kortaste vägar jämför *summor*:

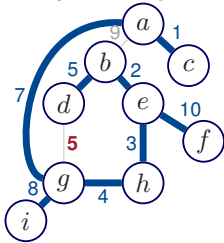
$$7 + 4 + 3 + 2 = 16 > 9, \text{ men}$$

$$7^2 + 4^2 + 3^2 + 2^2 = 78 < 81 = 9^2.$$

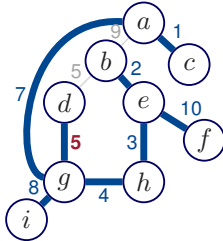
**Separation** Fast: grafen, ändringen · Varierar: vilket träd · Urskiljs: MST följer ordningen, kortaste väg summorna

# Är MST unikt?

Om  $d-g$  också väger 5 ...



... finns två MST, båda 40



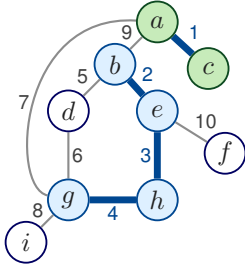
- Unika vikter  $\Rightarrow$  exakt ett MST. (Bevis: cykelegenskapen.)
- Lika vikter: flera MST kan finnas, men alla har samma *vikt*.
- Kruskal och Prim är fortfarande korrekta. Vilket träd de hittar beror på hur lika nycklar bryts.

**⚠ Missuppfattning:** "Prim och Kruskal gav olika träd, alltså har jag räknat fel." Bara om vikterna är unika.

**Generalisering** Fast: algoritmerna · Varierar: vikterna, från unika till lika · **Urskiljs:** "minimalt" betyder minsta vikt, inte ett visst träd

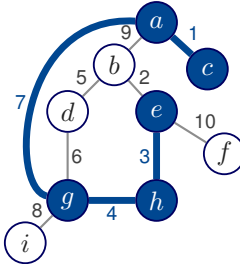
# Mitt i körningen: går det att se vilken algoritm det är?

**Kruskal, 4 kanter**



Två träd med kanter,  $\{a, c\}$  och  $\{b, e, g, h\}$ . Prim har alltid *ett* inte Prim.

**Prim, 4 kanter**



Ett träd med 7 men utan 2. Kruskal tar de lättaste kanterna som inte sluter en cykel, och 2 sluter ingen: inte Kruskal.

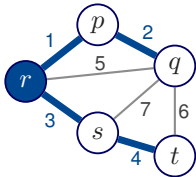
**Gemensam invariant:** de valda kanterna är en delmängd av något MST.

Här går det att se skillnad — av två *olika* skäl. Flera träd avslöjar Kruskal. En överhoppad billig kant avslöjar Prim.

Att det *alltid* går vore för starkt. Nästa bild.

**Kontrast** Fast: grafen, antal kanter · Varierar: algoritmen · Urskiljs: flera träd, eller en överhoppad billig kant

# Motexempel: Kruskal och Prim gör exakt samma sak



| Steg | Kruskal   | Prim från $r$ |
|------|-----------|---------------|
| 1    | $r-p$ (1) | $r-p$ (1)     |
| 2    | $p-q$ (2) | $p-q$ (2)     |
| 3    | $r-s$ (3) | $r-s$ (3)     |
| 4    | $s-t$ (4) | $s-t$ (4)     |

5, 6 och 7 sluter cykler. Samma tillstånd efter varje steg: Kruskals "skog" är hela tiden ett enda träd.

**Flera träd** med kanter  $\Rightarrow$  inte Prim. **Ett träd**  $\Rightarrow$  kan vara båda. (Prim från  $t$  börjar med 4.)

**⚠ Missuppfattning:** "Skog = Kruskal, träd = Prim." En skog kan vara ett enda träd — precis när Kruskals nästa kant hela tiden sitter i trädet runt startpunkten.

**Separation** Fast: algoritmerna · Varierar: grafen · Urskiljs: Prim ger alltid ett träd; Kruskal ibland också

# 2

## Förändrat flöde

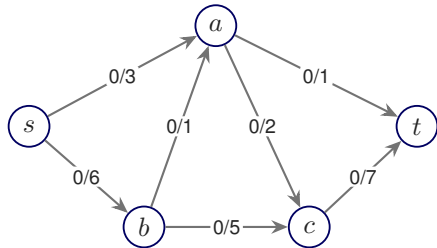
**Lärandeobjekt:** uppdatera ett maximalt flöde när en kapacitet ändras med 1 — i linjär tid, utan att börja om.

**Kritiska aspekter** (det vi kommer att variera):

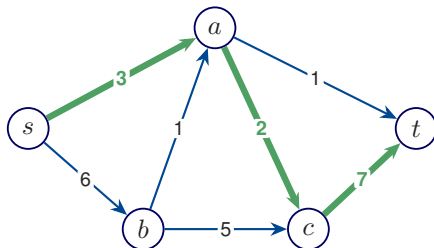
- ▶ Vad lösningen  $\Phi$  är: flödet på varje kant, inte bara värdet.
- ▶ Riktningen: ökning (högst +1) mot minskning (högst -1).
- ▶ Kanten: mättad eller inte — och ligger den i *alla* eller i *något* minsnitt?
- ▶ "Linjär tid": linjär i *vad*?

## Repetition: Ford–Fulkerson på exempelnätet

**Flöde  $f$**  (flöde/kapacitet, **röd** = mättad)



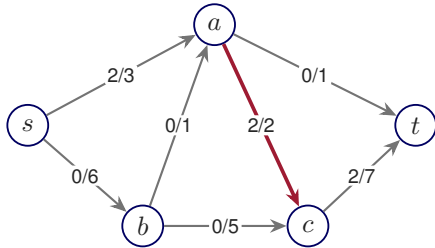
**Restflödesgraf  $G_f$**  (kvar, **ångra**)



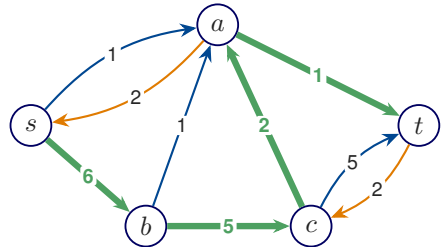
Inget flöde än: restgrafen är bara kapaciteterna. Välj en stig, vilken som helst:  $s \rightarrow a \rightarrow c \rightarrow t$ ,  
flaskhals  $\min(3, 2, 7) = 2$ .

# Repetition: Ford–Fulkerson på exempelnätet

**Flöde  $f$**  (flöde/kapacitet, **röd** = mättad)



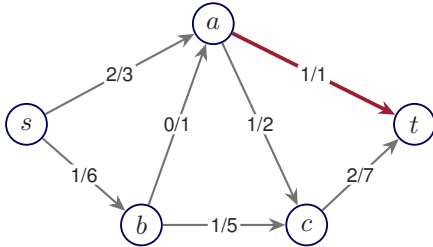
**Restflödesgraf  $G_f$**  (kvar, ångra)



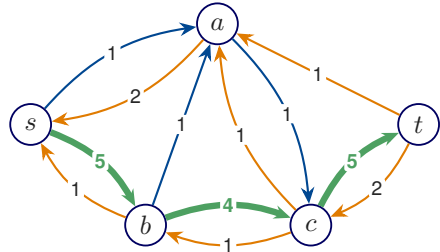
$|f| = 2$ . Bakåtkanten  $c \rightarrow a$  (orange) betyder "ångra flöde på  $a \rightarrow c$ ". Stig  $s \rightarrow b \rightarrow c \rightarrow a \rightarrow t$  använder den:  $+1$ .

# Repetition: Ford–Fulkerson på exempelnätet

**Flöde  $f$**  (flöde/kapacitet, **röd** = mättad)



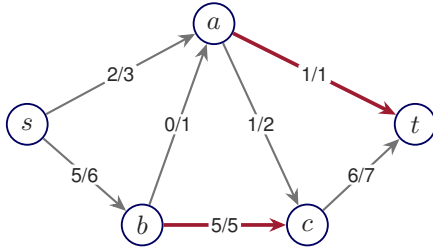
**Restflödesgraf  $G_f$**  (kvar, ångra)



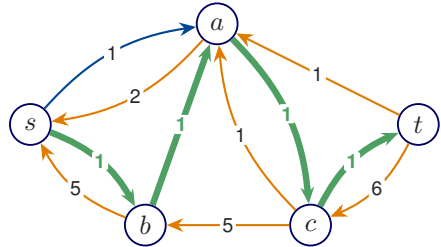
$|f| = 3$ . Stig  $s \rightarrow b \rightarrow c \rightarrow t$ , flaskhals  $\min(5, 4, 5) = 4$ .

# Repetition: Ford–Fulkerson på exempelnätet

**Flöde  $f$**  (flöde/kapacitet, **röd** = mättad)



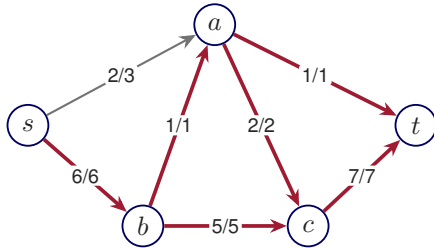
**Restflödesgraf  $G_f$**  (kvar, ångra)



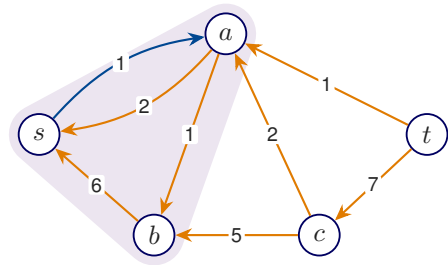
$|f| = 7$ . Stig  $s \rightarrow b \rightarrow a \rightarrow c \rightarrow t$ , flaskhals 1.

# Repetition: Ford–Fulkerson på exempelnätet

**Flöde  $f$**  (flöde/kapacitet, **röd** = mättad)

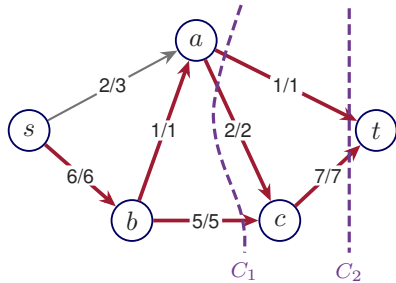


**Restflödesgraf  $G_f$**  (kvar, ångra)



$|f| = 8$ . Ingen stig  $s \rightsquigarrow t$  i  $G_f$ . Nåbara från  $s$ :  $\{s, a, b\}$  — ett minsnitt med kapacitet  $1 + 2 + 5 = 8$ .

# Repetition: maxflöde = minsnitt



Två minsnitt,  $C_1$  och  $C_2$ , båda 8.  
Att de är två kommer att spela roll.

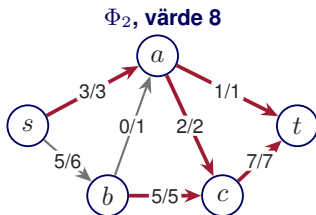
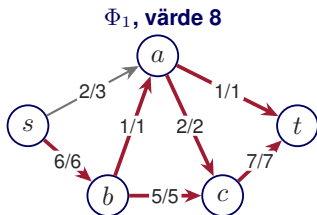
| $S$ ( $s$ -sidan) | kanter $S \rightarrow T$                            | kapacitet             |
|-------------------|-----------------------------------------------------|-----------------------|
| $\{s\}$           | $s \rightarrow a, s \rightarrow b$                  | $3 + 6 = 9$           |
| $\{s, a\}$        | $s \rightarrow b, a \rightarrow t, a \rightarrow c$ | $6 + 1 + 2 = 9$       |
| $\{s, b\}$        | $s \rightarrow a, b \rightarrow a, b \rightarrow c$ | $3 + 1 + 5 = 9$       |
| $\{s, c\}$        | $s \rightarrow a, s \rightarrow b, c \rightarrow t$ | $3 + 6 + 7 = 16$      |
| $\{s, a, b\}$     | $a \rightarrow t, a \rightarrow c, b \rightarrow c$ | $1 + 2 + 5 = 8 \ C_1$ |
| $\{s, a, c\}$     | $s \rightarrow b, a \rightarrow t, c \rightarrow t$ | $6 + 1 + 7 = 14$      |
| $\{s, b, c\}$     | $s \rightarrow a, b \rightarrow a, c \rightarrow t$ | $3 + 1 + 7 = 11$      |
| $\{s, a, b, c\}$  | $a \rightarrow t, c \rightarrow t$                  | $1 + 7 = 8 \ C_2$     |

**⚠ Missuppfattning:** "Snittets kapacitet är summan av kanterna som korsar det." Bara kanterna från  $S$  till  $T$  räknas. För  $S = \{s, a\}$  korsar  $b \rightarrow a$  snittet baklänges och räknas inte.

## Uppgift 2: Förändrat flöde

Anta att vi har en lösning  $\Phi$  till flödesproblemet för en graf. Beskriv en effektiv algoritm som hittar ett nytt maximalt flöde om kapaciteten längs en viss kant **a)** ökar med en enhet, **b)** minskar med en enhet. Tidskomplexiteten ska vara linjär, alltså  $O(|V| + |E|)$ .

**Vad består  $\Phi$  av?** Två maximala flöden i samma nät:

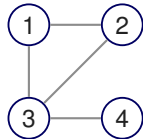


Samma värde. Olika flöden:  $s \rightarrow a$  2 mot 3,  $s \rightarrow b$  6 mot 5,  $b \rightarrow a$  1 mot 0.  $\Phi$  är flödet  $f(e)$  på varje kant. Med bara talet 8 finns inget att uppdatera — då återstår att räkna om från början.

**Kontrast** Fast: nätet och värdet 8 · **Varierar:** flödet på kanterna · **Urskiljs:** att  $\Phi$  är en funktion på kanterna, inte ett tal

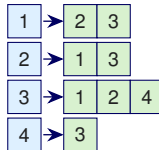
# Linjär i vad?

**Grafen**



$$|V| = 4, |E| = 5$$

**Grannlistor:**  $|V| + 2|E| = 12$  platser



**Grannmatris:**  $|V|^2 = 16$  platser

|   | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 0 |
| 2 | 1 | 0 | 1 | 0 |
| 3 | 1 | 1 | 0 | 1 |
| 4 | 0 | 0 | 1 | 0 |

- **Linjär tid** = högst en konstant gång *indatas storlek*. Man hinner läsa indata ett fast antal gånger, inte mer.
- BFS/DFS läser varje plats en gång:  $O(|V| + |E|)$  med grannlistor,  $O(|V|^2)$  med grannmatris. *Båda är linjära* — i storleken på sin representation.
- $O(|V| + |E|)$  är inte "linjärt i  $|V|$ ". I en fullständig graf är  $|E| = \binom{|V|}{2}$ , och då är  $O(|V| + |E|) = O(|V|^2)$ .

**Separation** Fast: grafen och algoritmen (BFS) · **Varierar:** representationen · **Urskiljs:** att "linjär" mäts i indatas storlek

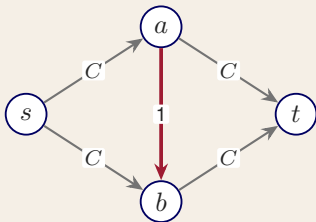
# Linjär i vad? Dagens uppgifter

| Uppgift           | Indatas storlek                  | "Linjärt"         | Kommentar                                     |
|-------------------|----------------------------------|-------------------|-----------------------------------------------|
| 2 Förändrat flöde | $ V  +  E $ (grafen och $\Phi$ ) | $O( V  +  E )$    | ett par grafsökningar                         |
| 3 Eulercykel      | $ V  +  E $                      | $O( V  +  E )$    | utdata har $ E  + 1$ hörn                     |
| 4 Julklappar      | $n$ listor à högst $m$ saker     | $O(n + nm)$       | ett varv av Ford–Fulkerson; det blir $n$ varv |
| 5 Sökning         | $n$ sorterade tal                | —                 | $\Theta(\log n)$ : läser inte ens all indata  |
| 6 Lampor          | $n$ lampor                       | $\Theta(n)$ byten | uppgiften vill ha exakt antal: $2n - 1$       |

**⚠ Missuppfattning:** "Eulercykeln hittas i  $O(n)$ ." (Så står det i äldre övningsbilder.) Om  $n = |V|$  är det fel: en graf med  $|V|$  hörn kan ha  $\Theta(|V|^2)$  kanter, och alla ska med i cykeln.

**Separation** Fast: ordet "linjär" · **Varierar:** storleksmåttet:  $|V| + |E|$ ,  $|V|^2$ ,  $n$ , antal byten · **Urskiljs:** att "linjär" alltid är linjär i något

## Kaninhål: Linjär i ett tal: pseudopolynomisk tid



$$C = 1\,000\,000 \approx 2^{20}$$

- ▶ Ford–Fulkerson väljer stigar godtyckligt. Med otur: varannan stig  $s \rightarrow a \rightarrow b \rightarrow t$ , varannan  $s \rightarrow b \rightarrow a \rightarrow t$  (baklänges på  $a \rightarrow b$ ).
- ▶ Varje stig ger  $+1$ . Maxflödet är  $2C$ : två miljoner varv.
- ▶ Indata: fem kanter, tal på cirka 20 bitar. Linjärt i *talet*  $C$ , exponentiellt i antalet *bitar*.
- ▶ Edmonds–Karp (kortaste stigen först) behöver två varv här, och  $O(|V| |E|^2)$  i allmänhet.

I uppgift 4 är alla kapaciteter 1 och  $|f^*| \leq n$ . Där är  $O(|E| \cdot |f^*|)$  helt ofarligt.

**Kontrast** Fast: nätet · Varierar: stigvalet: godtyckligt eller kortast · Urskiljs: linjärt i ett tal  $\neq$  linjärt i indatas längd

## 2a: kapaciteten på $(u, v)$ ökar med 1

- ▶  $\Phi$  är fortfarande ett tillåtet flöde: kapaciteten blev ju bara större.
- ▶ I restflödesgrafen  $G_\Phi$  ändras en enda sak: restkapaciteten  $u \rightarrow v$  ökar med 1.
- ▶ Kör **en** iteration av Ford–Fulkerson: sök en stig  $s \rightsquigarrow t$  i  $G_\Phi$ .
- ▶ Finns en stig: öka flödet med 1 längs den. Annars är  $\Phi$  fortfarande maximalt.

**Genväg:** var  $(u, v)$  inte mättad händer ingenting. Minsnittet som Ford–Fulkerson hittade (de nåbara hörnen) har bara mättade kanter ut, så dess kapacitet är oförändrad.

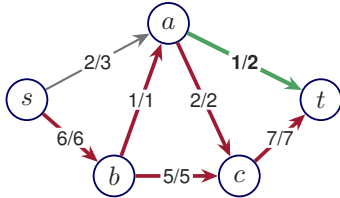
### ÖkaKapacitet( $G, \Phi, (u, v)$ )

- 1:  $c(u, v) \leftarrow c(u, v) + 1$
- 2: bygg  $G_\Phi$   $\triangleright O(|V| + |E|)$
- 3:  $P \leftarrow \text{BFS}(G_\Phi, s, t)$   $\triangleright O(|V| + |E|)$
- 4: **om**  $P$  finns **så**
- 5:     öka  $\Phi$  med 1 längs  $P$   $\triangleright O(|V|)$
- 6: **returnera**  $\Phi$

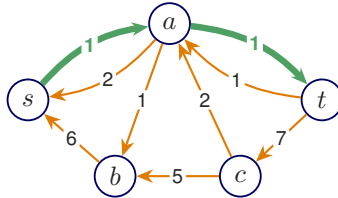
**⚠ Missuppfattning:** "Kör Ford–Fulkerson igen." Från början tar det  $O(|E| \cdot |f^*|)$  och kastar bort  $\Phi$ .

## 2a, exempel: $a \rightarrow t$ får kapacitet 2

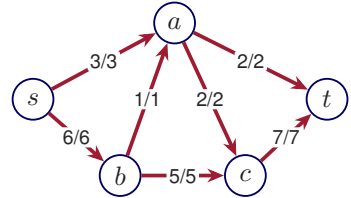
$\Phi_1$  med ny kapacitet



$G_\Phi$ : stigen  $s \rightarrow a \rightarrow t$



Nytt flöde, värde 9

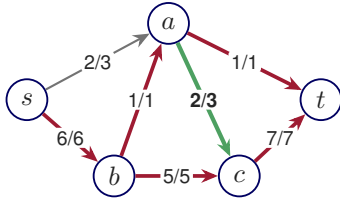


Restkanten  $a \rightarrow t$  (kapacitet 1) är ny, och  $s \rightarrow a$  hade redan 1 kvar. BFS hittar  $s \rightarrow a \rightarrow t$ . Kanten  $a \rightarrow t$  ligger i båda minsnitten  $C_1$  och  $C_2$ , så båda växte till 9.

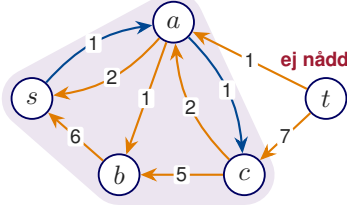
**Generalisering** Fast: metoden · Varierar: exemplet · Urskiljs: en sökning, en enhet

## 2a, exempel: $a \rightarrow c$ får kapacitet 3

$\Phi_1$  med ny kapacitet



$G_\Phi$ : nåbart från  $s$



Nu nås  $c$  via  $s \rightarrow a \rightarrow c$  — men  $c \rightarrow t$  är mättad.  $t$  nås inte.

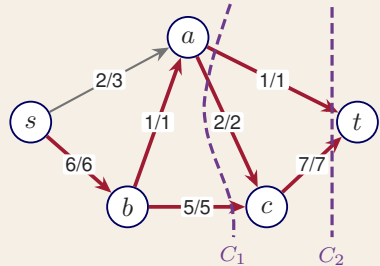
$\Phi_1$  är fortfarande maximalt: 8. Minsnittet  $C_2 = \{s, a, b, c\}$  innehåller inte  $a \rightarrow c$  och har kvar kapaciteten 8.

**⚠ Missuppfattning:** "Kanten var mättad, alltså hjälper det att öka den." Mättad är nödvändigt men inte tillräckligt.

**Kontrast Fast:** nätet och  $\Phi_1$  · **Varierar:** vilken mättad kant som ökas · **Urskiljs:** att en ökning kan vara verkningslös

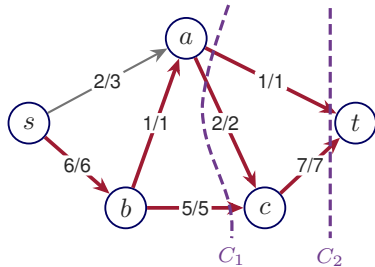
## Kaninhål: Varför räcker en enda sökning?

- ▶ Ett snitts kapacitet är summan av kanterna  $S \rightarrow T$ . När  $c(u, v)$  ökar med 1 växer varje snitt med 1 (om  $u \in S, v \in T$ ) eller med 0.
- ▶ Maxflöde = minsnitt, så det nya maxflödet är högst  $|\Phi| + 1$ .
- ▶ Allt är heltal: antingen samma eller exakt  $+1$ . En förbättrande stig, sedan är vi klara.
- ▶ Ingen stig? Då är  $\Phi$  maximalt. Det är Ford–Fulkersons eget stoppvillkor.



**Och om kapaciteten ökar med  $k$ ?** Då kan maxflödet öka med upp till  $k$ , och det kan behövas  $k$  sökningar:  $O(k(|V| + |E|))$ . Linjärt bara om  $k$  är en konstant.

## Samma ändring, olika kant: när hjälper +1?



| Kant              | $\Phi_1$ | mättad | i $C_1$ | i $C_2$ | +1 ger |
|-------------------|----------|--------|---------|---------|--------|
| $a \rightarrow t$ | 1/1      | ✓      | ✓       | ✓       | 9      |
| $a \rightarrow c$ | 2/2      | ✓      | ✓       |         | 8      |
| $b \rightarrow c$ | 5/5      | ✓      | ✓       |         | 8      |
| $c \rightarrow t$ | 7/7      | ✓      |         | ✓       | 8      |
| $s \rightarrow b$ | 6/6      | ✓      |         |         | 8      |
| $b \rightarrow a$ | 1/1      | ✓      |         |         | 8      |
| $s \rightarrow a$ | 2/3      |        |         |         | 8      |

Maxflödet ökar precis när kanten ligger i **alla** minsnitt. Ett enda minsnitt utan kanten håller taket kvar.

**⚠ Missuppfattning:** "Mättad kant  $\Rightarrow$  ökningen hjälper." Sex av sju kanter är mättade. En hjälper.

**Separation Fast:** nätet,  $\Phi_1$  och ändringen +1 · **Varierar:** kanten · **Urskiljs:** att minsnitten bestämmer, inte mättnaden

## 2b: kapaciteten på $(u, v)$ minskar med 1

**Fall 1:**  $(u, v)$  är inte mättad.  $\Phi$  är fortfarande tillåtet, och maxflödet kan inte öka när en kapacitet minskar. Klart.

**Annars:** minska  $\Phi(u, v)$  med 1. Nu kommer det in en enhet mer till  $u$  än som går ut, och en enhet mindre till  $v$ .

**Fall 2:** finns en stig  $u \rightsquigarrow v$  i  $G_\Phi$ ? Led enheten den vägen. Värdet är oförändrat.

**Fall 3:** annars: skicka tillbaka enheten,  $u \rightsquigarrow s$ , och ta bort en enhet från  $v$  till  $t$  genom att gå  $t \rightsquigarrow v$ . Värdet minskar med 1.

### MinskaKapacitet( $G, \Phi, (u, v)$ )

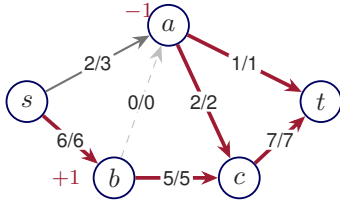
- 1:  $c(u, v) \leftarrow c(u, v) - 1$
- 2: **om**  $\Phi(u, v) \leq c(u, v)$  **så**
- 3:     **returnera**  $\Phi$  ▷ fall 1
- 4:  $\Phi(u, v) \leftarrow \Phi(u, v) - 1$  ▷ obalans i  $u, v$
- 5: **om** stig  $P: u \rightsquigarrow v$  finns i  $G_\Phi$  **så**
- 6:     öka  $\Phi$  med 1 längs  $P$  ▷ fall 2
- 7: **annars**
- 8:     öka  $\Phi$  med 1 längs en stig  $u \rightsquigarrow s$  ▷ fall 3
- 9:     öka  $\Phi$  med 1 längs en stig  $t \rightsquigarrow v$
- 10: **returnera**  $\Phi$

Högst tre grafsökningar:  $O(|V| + |E|)$ .

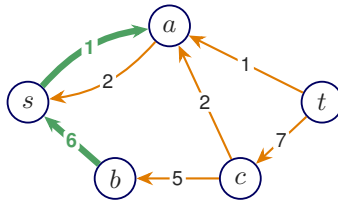
**⚠ Missuppfattning:** "Minska flödet på kanten med 1, klart." Då kommer mer in i  $u$  än som går ut. Det är inte längre ett flöde.

## 2b, exempel (fall 2): $b \rightarrow a$ får kapacitet 0

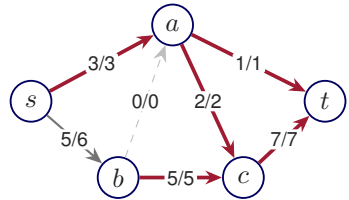
Obalans efter minskningen



$G_\Phi$ : stigen  $b \rightarrow s \rightarrow a$



Nytt flöde, värde 8

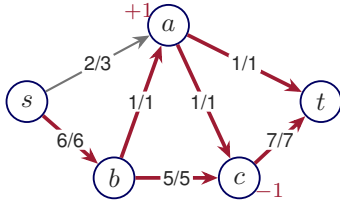


$b$  har en enhet för mycket,  $a$  en för lite. Stigen  $b \rightarrow s \rightarrow a$  går baklänges på  $s \rightarrow b$  och framlänges på  $s \rightarrow a$ : enheten tar vägen  $s \rightarrow a$  i stället. Resultatet är  $\Phi_2$  — det andra maxflödet från bilden om vad  $\Phi$  är.

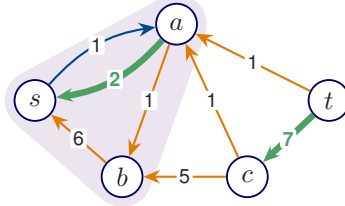
**Kontrast** Fast: nätet,  $\Phi_1$  och minskningen · **Varierar**: kanten (här: ingen i något minschnitt) · **Urskiljs**: omledning bevarar värdet

## 2b, exempel (fall 3): $a \rightarrow c$ får kapacitet 1

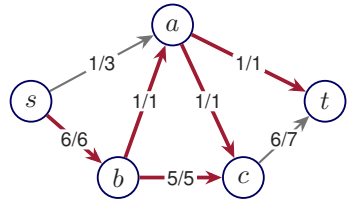
Obalans efter minskningen



$G_\Phi$ : nåbart från  $a$



Nytt flöde, värde 7



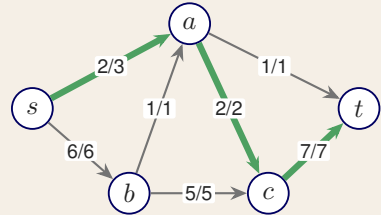
Från  $a$  nås bara  $\{a, s, b\}$ , inte  $c$ . Alltså fall 3: skicka tillbaka längs  $a \rightarrow s$  och ta bort en enhet längs  $t \rightarrow c$  (båda baklänges i restgrafen, gröna). Kanten  $a \rightarrow c$  ligger i minsnittet  $C_1$ , som nu har kapacitet 7.

**Kontrast Fast:** nätet,  $\Phi_1$  och minskningen · **Varierar:** kanten (här: i minsnittet  $C_1$ ) · **Urskiljs:** när värdet måste sjunka

## **Kaninhål:** Varför finns stigarna $u \rightsquigarrow s$ och $t \rightsquigarrow v$ ?

- **Flödesuppdelning:** varje flöde är en summa av  $s$ - $t$ -stigarna och cykler med positivt flöde.  $\Phi_1: s \rightarrow b \rightarrow c \rightarrow t$  (5),  $s \rightarrow a \rightarrow c \rightarrow t$  (2),  $s \rightarrow b \rightarrow a \rightarrow t$  (1).
- $(u, v)$  var mättad, så  $\Phi(u, v) \geq 1$ : någon stig eller cykel går genom  $(u, v)$ .
- **Stig**  $s \rightsquigarrow u \rightarrow v \rightsquigarrow t$ : flödet på delarna ger bakåtkanter  $u \rightsquigarrow s$  och  $t \rightsquigarrow v$  i  $G_\Phi$ .
- **Cykel** genom  $(u, v)$ : resten av cykeln, baklänges, är en stig  $u \rightsquigarrow v$ . Då var vi i fall 2.

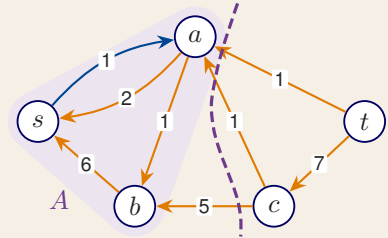
$s \rightarrow a \rightarrow c \rightarrow t$  bär 2 enheter



Gör sökningarna en i taget: först  $u \rightsquigarrow s$ , uppdatera  $\Phi$ , sedan  $t \rightsquigarrow v$  i den nya restgrafen. Samma uppdelningsargument på det uppdaterade flödet visar att den andra stigen fortfarande finns.

## Kaninhål: Är flödet i fall 3 säkert maximalt?

- ▶ Låt  $A$  = hörnen som nås från  $u$  i  $G_\Phi$ . Fall 3 betyder  $v \notin A$ .
- ▶  $s \in A$ , för stigen  $u \rightsquigarrow s$  finns.  $t \notin A$ , annars fanns  $u \rightsquigarrow t \rightsquigarrow v$ .
- ▶ Ingen restkant lämnar  $A$ : kanterna ut ur  $A$  är mättade, kanterna in är tomma.
- ▶ Flödet över snittet är då lika med dess kapacitet, och det är  $|\Phi| - 1$ : enheten som blev över i  $u$  stannar i  $A$ .
- ▶ Tillbakatrycken sker helt inom  $A$  respektive helt utanför. Nya flödet har värdet  $|\Phi| - 1 = \text{ett snitts kapacitet}$ . Alltså maximalt.



Här:  $A = \{s, a, b\}$ , och  $1 + 1 + 5 = 7$ .

Ingen extra kontroll behövs. (En extra BFS skulle inte förstöra linjäriteten. Den är bara onödig.)

## **Kaninhål: Alternativ lösning: b) via a)**

Om  $(u, v)$  är mättad:

- 1. Ta bort en enhet flöde genom  $(u, v)$ :** minska  $\Phi(u, v)$  och tryck tillbaka längs  $u \rightsquigarrow s$  och  $t \rightsquigarrow v$ . Nu är  $\Phi$  tillåtet med den nya kapaciteten och har värdet  $|\Phi| - 1$ .
- 2. Gör som i a):** en sökning  $s \rightsquigarrow t$ . Maxflödet är nu  $|\Phi| - 1$  eller  $|\Phi|$ , så en förbättrande stig räcker.

Exempel  $b \rightarrow a$ : steg 1 tar bort enheten på  $s \rightarrow b \rightarrow a \rightarrow t$  (värde 7), steg 2 hittar  $s \rightarrow a \rightarrow t$  (värde 8). Samma slutresultat,  $\Phi_2$ .

Tre sökningar i stället för en eller två — fortfarande  $O(|V| + |E|)$ . Vinsten är att korrekthetsbeviset blir kortare: efter steg 1 är vi exakt i situationen i a).

**Generalisering** **Fast:** metoden: restgraf och en sökning · **Varierar:** problemet: öka eller minska · **Urskiljs:** att b) är "ta bort en enhet" följt av a)

## Öka och minska, kant för kant

| Kant              | $\Phi_1$ | i $C_1$ | i $C_2$ | +1 | -1 | fall vid -1                        |
|-------------------|----------|---------|---------|----|----|------------------------------------|
| $a \rightarrow t$ | 1/1      | ✓       | ✓       | 9  | 7  | 3                                  |
| $a \rightarrow c$ | 2/2      | ✓       |         | 8  | 7  | 3                                  |
| $b \rightarrow c$ | 5/5      | ✓       |         | 8  | 7  | 3                                  |
| $c \rightarrow t$ | 7/7      |         | ✓       | 8  | 7  | 3                                  |
| $s \rightarrow b$ | 6/6      |         |         | 8  | 8  | 2: $s \rightarrow a \rightarrow b$ |
| $b \rightarrow a$ | 1/1      |         |         | 8  | 8  | 2: $b \rightarrow s \rightarrow a$ |
| $s \rightarrow a$ | 2/3      |         |         | 8  | 8  | 1                                  |

+1 ökar maxflödet  $\iff$  kanten ligger i **alla** minsnitt.

-1 minskar maxflödet  $\iff$  kanten ligger i **något** minsnitt.

**⚠ Missuppfattning:** "Ökning och minskning är symmetriska."  $a \rightarrow c$  kan inte öka maxflödet, men kan minska det.

**⚠ Missuppfattning:** "Mättad är en egenskap hos kanten." Hos kanten *och*  $\Phi: s \rightarrow b$  är mättad i  $\Phi_1$  (6/6) men inte i  $\Phi_2$  (5/6).

**Fusion Fast:** nätet · **Varierar:** kanten och riktningen samtidigt · **Urskiljs:** alla mot något minsnitt; mättnad beror på  $\Phi$

## Vanliga fel i uppgift 2

⚠ **Missuppfattning:** " $\Phi$  är ett tal, 8." Det är flödet på varje kant. Med bara talet måste man räkna om från början.

⚠ **Missuppfattning:** "Öka flödet på  $(u, v)$  med 1." Då bryts flödesbevarandet i  $u$  och  $v$ , och enheten kanske inte ens kan komma fram till  $t$ .

⚠ **Missuppfattning:** "Minska flödet på  $(u, v)$  med 1, klart." Samma fel åt andra hållet. Obalansen måste ledas om (fall 2) eller skickas tillbaka (fall 3).

⚠ **Missuppfattning:** "Linjär tid, alltså  $O(|V|)$ ." Linjärt i indatas storlek, här  $|V| + |E|$ . Se "Linjär i vad?".

Gemensam nämnare för de rätta lösningarna: ändra i restgraf, gör en grafsökning, och låt maxflöde = min-snitt tala om att det räcker.

**Kontrast** Fast: uppgiften · **Varierar:** lösningsidén, rätt och nästan rätt · **Urskiljs:** vad som gör de rätta rätt

# 3

## Eulercykel

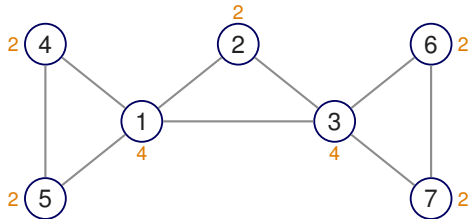
**Lärandeobjekt:** hitta en Eulercykel i linjär tid — och kunna säga vad "linjär" betyder här.

**Kritiska aspekter** (det vi kommer att variera):

- ▶ Varje *kant* exakt en gång. (Varje *hörn* en gång är ett helt annat problem.)
- ▶ Varför jämnt gradtal räcker: man kan bara fastna där man började.
- ▶ Linjärt i  $|V| + |E|$  kräver rätt datastruktur: delade kanter och en pekare per hörn.

## Uppgift 3: Eulercykel

Givet en oriktad sammanhängande graf  $G = \langle V, E \rangle$  där alla hörn har jämnt gradtal. Hitta en Eulercykel, dvs. en sluten stig som passerar varje kant i  $E$  exakt en gång. Algoritmen ska gå i linjär tid.

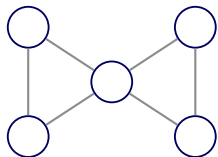


gradtal i orange

- ▶ Hörn får upprepas: här måste 1 och 3 passeras två gånger var.
- ▶ Villkoren är nödvändiga: varje gång cykeln går *in* i ett hörn går den också *ut*, så gradtalen blir jämna.
- ▶ Att de också är tillräckliga visar algoritmen.

# Euler eller Hamilton? Ett ord skiljer

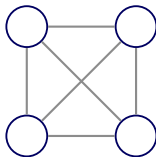
Fluga



Euler: ✓ Hamilton: ✗

Mitten måste passeras två gånger.

$K_4$



Euler: ✗ Hamilton: ✓

Alla gradtal är 3.

**Euler:** varje *kant* exakt en gång.  
Linjär tid.

**Hamilton:** varje *hörn* exakt en gång.  
NP-fullständigt.

Nästan samma mening. Helt olika problem.

**⚠ Missuppfattning:** "En Eulercykel besöker alla hörn en gång." Det är en Hamiltoncykel — och för den finns ingen känd polynomisk algoritm.

**Kontrast Fast:** "besök allt exakt en gång" · **Varierar:** kanter eller hörn · **Urskiljs:** vad ordet "varje" syftar på

## Idén: en hare och en sköldpadda

### Haren (Viggos PATHFINDER)

Springer från ett hörn  $u$  längs omärkta kanter, märker dem och stannar först när den är tillbaka i  $u$ . Jämna gradtal gör att den bara kan fastna där.

### Sköldpaddan (Viggos STRAGGLER)

Går efter längs harens stig och skriver ut hörnen. Har ett hörn omärkta kanter kvar skickar den ut haren därifrån — och går den nya tur *innan* den fortsätter.

Varje tur är en sluten stig. Sköldpaddan skarvar in turerna i varandra, precis där de börjar. Ingen kant springs två gånger av haren, och ingen kant går två gånger av sköldpaddan.

Därför linjärt: varje kant märks en gång av haren och passeras en gång av sköldpaddan. (Förutsett att båda hittar nästa omärkta kant i konstant tid — mer om det strax.)

# Pseudokod

## EulerCykel( $G$ )

- 1:  $cykel \leftarrow [v_0]$
- 2: välj en kant  $(v_0, t)$  och märk den
- 3: SKÖLDPADDA(HARE( $v_0, t$ ))
- 4: **returnera**  $cykel$

## Hare( $start, cur$ )

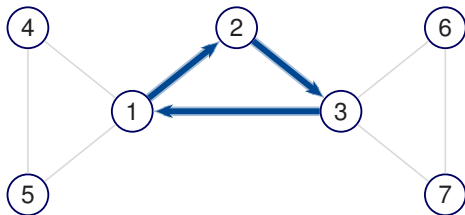
- 1:  $stig \leftarrow [cur]$
- 2: **medan**  $cur \neq start$  **gör**
- 3:     välj en omärkt kant  $(cur, v)$ ; märk den
- 4:     lägg  $v$  sist i  $stig$ ;  $cur \leftarrow v$
- 5: **returnera**  $stig$

## Sköldpadda( $stig$ )

- 1: **för alla**  $u$  i  $stig$ , i ordning **gör**
- 2:     lägg  $u$  sist i  $cykel$
- 3:     **för alla** kanter  $(u, v)$  **gör**
- 4:         **om**  $(u, v)$  är omärkt **så**
- 5:             märk  $(u, v)$
- 6:             SKÖLDPADDA(HARE( $u, v$ ))

Omärkt-testet görs om för varje kant: det rekursiva anropet kan ha märkt kanter som slingan ännu inte kommit till.

# Eulercykeln steg för steg



tur 1   tur 2   tur 3   ○ sköldpaddan

Senaste tur (haren):  $[2, 3, 1]$

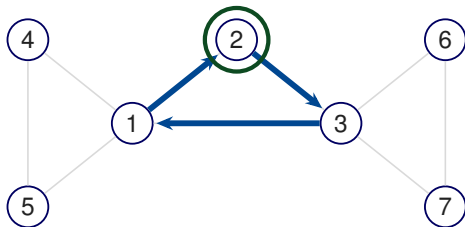
Cykel (sköldpaddan):

$cykel = [1]$

Märk (1, 2). **Haren** springer längs omärkta kanter:  $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ . Den fastnar i 1 — starthörnet.

**Generalisering** **Fast:** algoritmen · **Varierar:** var turerna börjar · **Urskiljs:** att turer skarvas in där sköldpaddan hittar omärkta kanter

# Eulercykeln steg för steg



tur 1 tur 2 tur 3 ○ sköldpaddan

Senaste tur (haren):  $[2, 3, 1]$

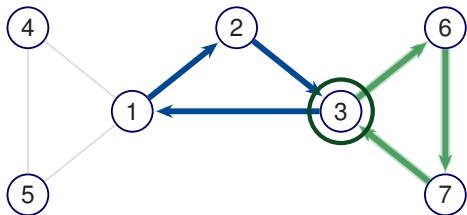
Cykel (sköldpaddan):

$cykel = [1, 2]$

**Sköldpaddan** går efter längs stigen. I 2: alla kanter märkta. Gå vidare.

**Generalisering** Fast: algoritmen · Varierar: var turerna börjar · Urskiljs: att turer skarvas in där sköldpaddan hittar omärkta kanter

# Eulercykeln steg för steg



tur 1   tur 2   tur 3   ○ sköldpaddan

Senaste tur (haren):  $[6, 7, 3]$

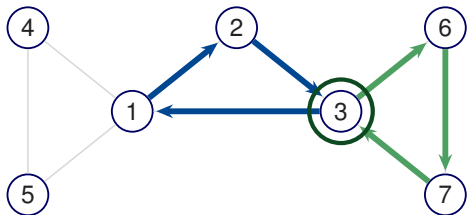
Cykel (sköldpaddan):

$cykel = [1, 2, 3]$

I 3: omärkta kanter kvar! Skicka ut haren från 3:  $3 \rightarrow 6 \rightarrow 7 \rightarrow 3$ .

**Generalisering** Fast: algoritmen · Varierar: var turerna börjar · Urskiljs: att turer skarvas in där sköldpaddan hittar omärkta kanter

# Eulercykeln steg för steg



tur 1   tur 2   tur 3   ○ sköldpaddan

Senaste tur (haren): [6, 7, 3]

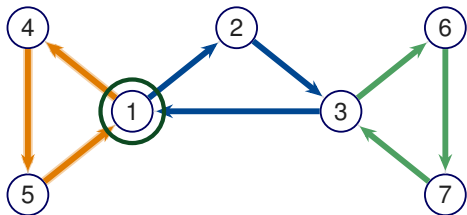
Cykel (sköldpaddan):

$cykel = [1, 2, 3, 6, 7, 3]$

Sköldpaddan går den nya turen *innan* den fortsätter: 6, 7, 3.

**Generalisering** Fast: algoritmen · Varierar: var turerna börjar · Urskiljs: att turer skarvas in där sköldpaddan hittar omärkta kanter

# Eulercykeln steg för steg



tur 1   tur 2   tur 3   ○ sköldpaddan

Senaste tur (haren):  $[4, 5, 1]$

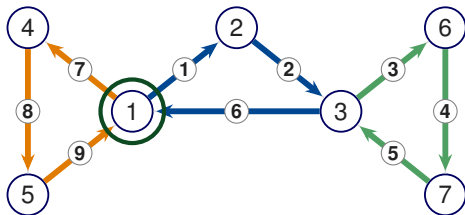
Cykel (sköldpaddan):

*cykel* =  $[1, 2, 3, 6, 7, 3, \mathbf{1}]$

Tillbaka på första turen. I 1: omärkta kanter.  
Haren:  $1 \rightarrow 4 \rightarrow 5 \rightarrow 1$ .

**Generalisering** Fast: algoritmen · Varierar: var turerna börjar · Urskiljs: att turer skarvas in där sköldpaddan hittar omärkta kanter

# Eulercykeln steg för steg



tur 1   tur 2   tur 3   ○ sköldpaddan

Senaste tur (haren):  $[4, 5, 1]$

Cykel (sköldpaddan):

$cykel = [1, 2, 3, 6, 7, 3, 1, 4, 5, 1]$

Sköldpaddan går 4, 5, 1. Klart:  $10 = |E| + 1$  hörn. Siffrorna visar cykelns ordning.

**Generalisering** Fast: algoritmen · Varierar: var turerna börjar · Urskiljs: att turer skarvas in där sköldpaddan hittar omärkta kanter

## **Kaninhål: Varför fastnar haren bara i starthörnet?**

- ▶ Varje gång haren kommer *in* i ett hörn  $w \neq u$  har den använt ett udda antal av  $w$ 's kanter (*in*, *ut*, *in*, *ut*, ..., *in*).
- ▶  $\deg(w)$  är jämnt, så minst en kant är omärkt. Haren kommer alltid vidare.
- ▶ I  $u$  är det tvärtom: den första kanten gick *ut*, så efter varje ankomst är ett jämnt antal använt. Där kan den fastna.
- ▶ En sluten tur tar bort ett jämnt antal kanter från varje hörn. Alla gradtal är fortfarande jämna, och argumentet gäller nästa tur också.

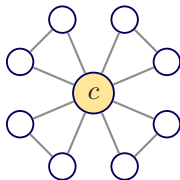
**Och varför kommer alla kanter med?** Anta att en kant blev omärkt. Grafen är sammanhängande, så någon omärkt kant sitter i ett hörn som finns i cykeln. Men sköldpaddan kontrollerade varje hörn i cykeln. Motsägelse.

**Udda gradtal:** har exakt två hörn udda grad finns en Eulerväg mellan dem. Lägg till en extra kant mellan dem, hitta en cykel, ta bort kanten.

# Linjär i vad? Tre detaljer som avgör

**Indata:** grannlistor,  $|V| + |E|$ . **Utdata:**  $|E| + 1$  hörn, så  $\Omega(|E|)$  krävs ändå.

1. **En kant, två listor.**  $\{u, v\}$  står i både  $u$ 's och  $v$ 's lista. Lagra kanten en gång, med markeringen; båda listorna pekar på den.
2. **En pekare per hörn** till nästa ej undersökta plats i listan. Utan den börjar haren och sköldpaddan om från början varje gång.
3. Med båda: varje listplats passeras ett konstant antal gånger.  $O(|V| + |E|)$ .



Blomma med  $k$  blad:  $\deg(c) = 2k$ , och cykeln passerar  $c$   $k$  gånger. Utan pekare:  $k \cdot 2k = \Theta(|E|^2)$ .

**⚠ Missuppfattning:** "Eulercykeln hittas i  $O(n)$ ." Med  $n = |V|$  är det omöjligt:  $K_{101}$  har 101 hörn och 5 050 kanter, och alla ska med i cykeln.

**Separation** Fast: algoritmen · **Varierar:** datastrukturen: delade kanter, pekare · **Urskiljs:** att "linjär" kräver rätt representation

## Kaninhål: Utan rekursion: Hierholzer med stack

### Hierholzer( $G, v_0$ )

```

1:  $S \leftarrow [v_0]; \text{cykel} \leftarrow []$ 
2: medan  $S \neq \emptyset$  gör
3:    $u \leftarrow$  översta elementet i  $S$ 
4:   om  $u$  har en omärkt kant  $(u, v)$  så           ▷ pekaren
5:     märk  $(u, v)$ ; lägg  $v$  överst i  $S$ 
6:   annars
7:     ta bort  $u$  ur  $S$ ; lägg  $u$  sist i  $\text{cykel}$ 
8: returnera  $\text{cykel}$ 

```

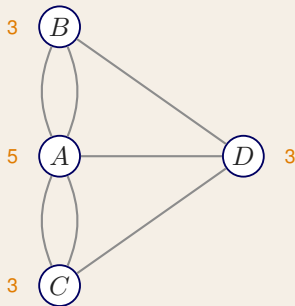
Stacken är harens stig. När haren fastnar backar vi och skriver ut hörnen.

På exemplet:  $[1, 5, 4, 1, 3, 7, 6, 3, 2, 1]$  — samma cykel som förut, baklänges. En Eulercykel baklänges är också en Eulercykel.

**Varför bry sig?** Rekursionen i hare/sköldpadda kan bli  $\Theta(|E|)$  djup. Med en miljon kanter tar anropsstacken slut i Java och Python långt innan tiden gör det.

**Kontrast: Fleury (1883).** Gå aldrig över en bro (en kant vars borttagning delar grafen) om det finns ett annat val. Korrekt, men varje steg kräver en brokontroll:  $O(|E|^2)$ .

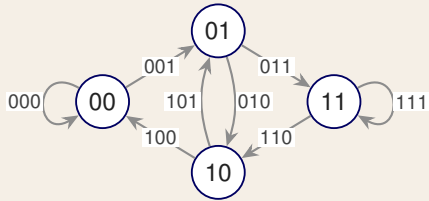
## Kaninhål: Königsberg 1736: där allt började



- Fyra landområden, sju broar. Går det att promenera över varje bro exakt en gång?
- Euler: nej. Alla fyra hörnen har udda grad (5, 3, 3, 3). För en cykel får inget ha udda grad, för en väg högst två.
- Eulers observation räknas som grafteorins första sats. Han bevisade bara att villkoret är nödvändigt. Att det räcker visades senare (Hierholzer, 1873).

*A* är ön Kneiphof, *B* och *C* flodstränderna, *D* området öster om ön. Euler behövde aldrig promenera: det räckte att räkna gradtal.

## Kaninhål: Kodlåset: Eulercykler i riktade grafer



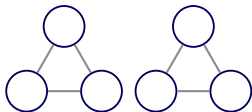
- ▶ Ett kodlås öppnas så fort de fyra senaste tryckningarna är rätt. Hur många tryck krävs för att pröva alla  $10^4$  koder?
- ▶ Hörn: tre siffror. Kant: en fjärde. Ingrad = utgrad = 10 överallt, så en riktad Eulercykel finns.
- ▶ Svar:  $10^4 + 3 = 10\,003$  tryck i stället för 40 000.
- ▶ Binärt, tre siffror (bilden): 0001011100 innehåller alla åtta.

Samma algoritim, riktade grafer: de Bruijn-följder. (Kodlås borde alltså kräva Enter.)

**Generalisering** Fast: algoritmen · Varierar: grafen: oriktad eller riktad · Urskiljs: att villkoret är "lika mycket in som ut"

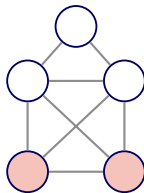
## Båda villkoren behövs

Jämna grader, två delar



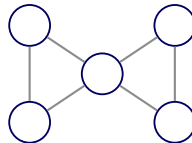
Ingen Eulercykel.

Sammanhängande, två udda



Nikolaus hus: Eulerväg mellan de röda, men ingen cykel.

Sammanhängande, jämna



Eulercykel finns.

**⚠ Missuppfattning:** "Alla grader jämna räcker." Två trianglar utan förbindelse har jämna grader och ingen Eulercykel. (Isolerade hörn gör däremot inget: de har inga kanter att passera.)

**Separation** Fast: frågan · **Varierar:** ett villkor i taget · **Urskiljs:** att både sammanhang och jämna grader behövs

## Vanliga fel i uppgift 3

**⚠ Missuppfattning:** "Gå tills du fastnar, klart." Första turen  $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$  tar 3 av 9 kanter. Man fastnar i starthörnet, inte när grafen är slut.

**⚠ Missuppfattning:** "Märk kanten i  $u$ 's lista." Den står i  $v$ 's lista också. Märks bara ena riktningen går kanten två gånger.

**⚠ Missuppfattning:** "Hitta alla turer och lägg dem efter varandra." Tur 2 börjar i 3 och tur 3 i 1. Varje tur måste skarvas in där den börjar, annars blir det ingen sammanhängande promenad.

**⚠ Missuppfattning:** "Linjär tid:  $O(|V|)$ ." Svaret har  $|E| + 1$  hörn. Linjärt betyder  $O(|V| + |E|)$  — och det kräver en pekare per hörn.

**Kontrast** Fast: uppgiften · **Varierar:** lösningsidén, rätt och nästan rätt · **Urskiljs:** fastna, skarva, märka, räkna

# 4

## Julklappsfördelning

**Lärandeobjekt:** modellera en tilldelning som bipartit matchning, lösa den med flöde — och ange tiden i väldefinierade storheter.

**Kritiska aspekter** (det vi kommer att variera):

- ▶ Modellen: vad är hörn, kanter och kapaciteter — och varför just 1?
- ▶ Girigt räcker inte: en förbättrande stig kan ta tillbaka det som redan delats ut.
- ▶ Vad  $m$  betyder avgör vad  $O(\dots)$  blir.

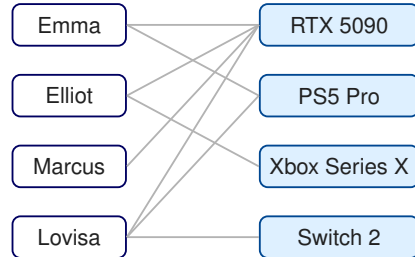
## Uppgift 4: Julklappsfordelning

En pappa ska ge sina  $n$  barn var sin julklapp. Varje barn har skrivit en önskelista. Pappan vill ge varje barn en julklapp från barnets önskelista, men inte samma julklapp till flera barn. Konstruera och analysera en effektiv algoritm. Du kan anta att det finns högst  $m$  saker på varje önskelista.

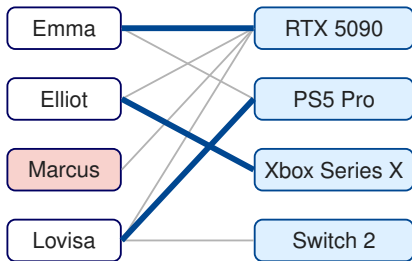
**Årets instans.** Tomten ska ge övningsassistenterna var sin klapp.

|        |                             |
|--------|-----------------------------|
| Emma   | RTX 5090, PS5 Pro           |
| Elliot | RTX 5090, Xbox Series X     |
| Marcus | RTX 5090                    |
| Lovisa | RTX 5090, PS5 Pro, Switch 2 |

Alla vill ha Nvidia-kortet. Det finns ett.



## Första försöket: girigt



**⚠ Missuppfattning:** "Ge var och en första lediga sak." Resultatet beror på ordningen, och ett tidigt val kan behöva ångras.

I listordning, var och en tar första lediga sak:  
 Emma tar RTX-kortet. Elliot tar Xboxen.  
 Marcus vill bara ha RTX-kortet, som är borta.  
 Lovisa tar PS5:an.

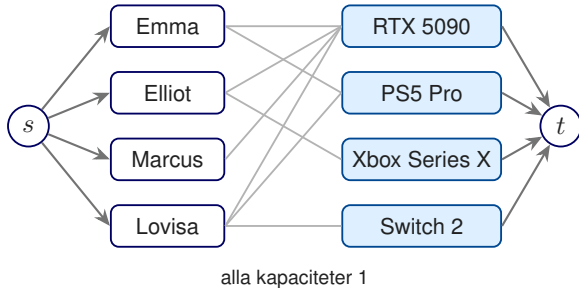
Tre klappar. Marcus får ingenting.

(Marcus har gjort de här bilderna. Han har noterat utfallet.)

**Maximal** (går inte att utöka)  $\neq$  **största** (flest par).  
 Den giriga är maximal. Största har fyra par.

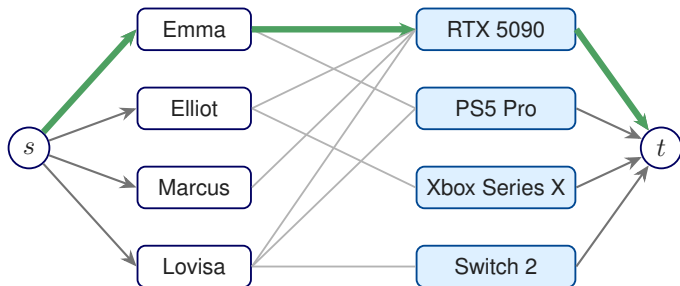
**Kontrast** **Fast:** instansen · **Varierar:** algoritmen: girig eller flöde · **Urskiljs:** att ett lokalt val kan behöva ångras

## Modellen: matchning som flöde



- ▶  $s \rightarrow$  barn, kapacitet 1: ingen får två klappar.
- ▶ barn  $\rightarrow$  sak om saken står på listan.
- ▶ sak  $\rightarrow t$ , kapacitet 1: ingen sak delas ut två gånger.
- ▶ Heltalskapaciteter  $\Rightarrow$  Ford–Fulkerson ger ett *heltalsflöde*. Varje kant bär 0 eller 1, och kanterna i mitten med flöde 1 är matchningen.
- ▶ Maxflöde = största matchning.

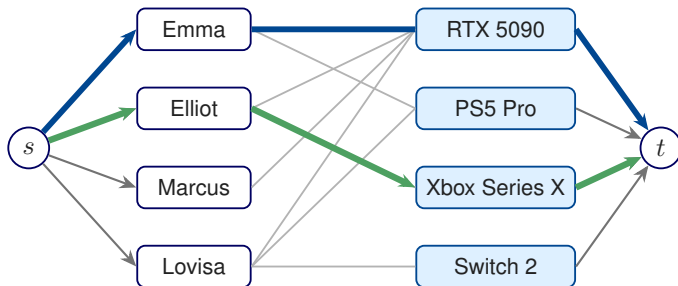
## Ford–Fulkerson (med BFS) på julklapparna



matchat förbättrande stig bakåtkant: ångra

Alla kapaciteter är 1. BFS hittar  $s \rightarrow \text{Emma} \rightarrow \text{RTX} \rightarrow t$ . Emma får grafikkortet. Tills vidare.

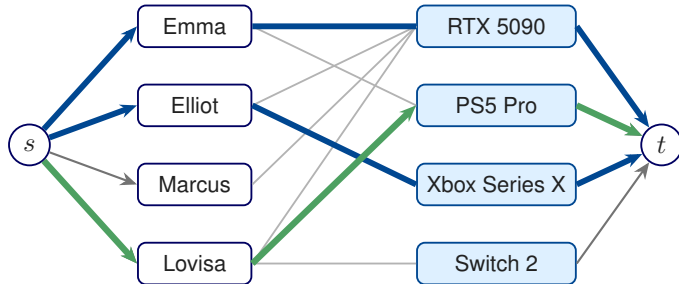
## Ford–Fulkerson (med BFS) på julklapparna



matchat   förbättrande stig   bakåtkant: ångra

$s \rightarrow \text{Elliot} \rightarrow \text{Xbox} \rightarrow t$ : RTX-kortet är upptaget, så Elliot tar sitt andra val.

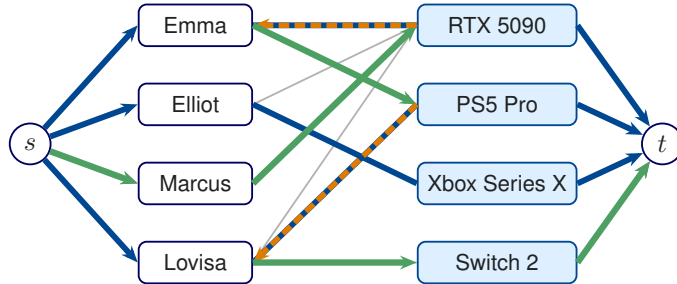
# Ford–Fulkerson (med BFS) på julklapparna



matchat förbättrande stig bakåtkant: ångra

$s \rightarrow \text{Lovisa} \rightarrow \text{PS5} \rightarrow t$ . Tre av fyra har en klapp. Marcus står kvar tomhänt.

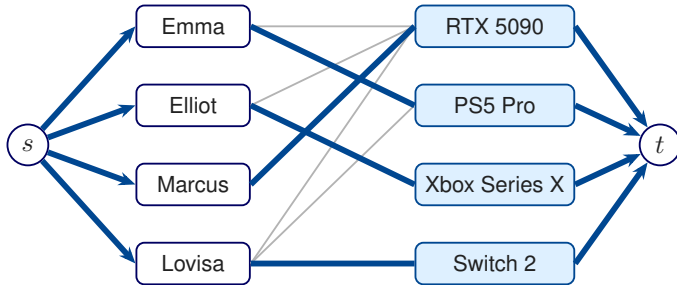
# Ford–Fulkerson (med BFS) på julklapparna



matchat   förbättrande stig   bakåtkant: ångra

Enda stigen: Marcus tar RTX från Emma, Emma tar PS5 från Lovisa, Lovisa tar Switch 2. Två bakåtkanter (streckade).

## Ford–Fulkerson (med BFS) på julklapparna

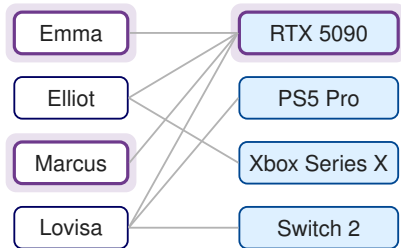


matchat förbättrande stig bakåtkant: ångra

Flöde 4 = perfekt matchning. Stigen i matchningstermer: alternerande ny, bort, ny, bort, ny.

## När räcker klapparna inte? Halls villkor

Om Emma också bara vill ha RTX-kortet



Emma och Marcus önskar tillsammans en enda sak. Två personer, ett kort. Maxflödet blir 3.

Minsnittet bevisar det:  $s \rightarrow \text{Elliot}$ ,  $s \rightarrow \text{Lovisa}$ ,  $\text{RTX} \rightarrow t$ . Kapacitet 3.

Vill alla bara ha RTX-kortet blir maxflödet 1: minst tre besvikna övningsassistenter.

**Halls sats:** alla kan få en klapp  $\iff$  varje grupp  $S$  av barn önskar tillsammans minst  $|S|$  olika saker. Annars finns en grupp som bevisar det, och minsnittet hittar den.

**Separation Fast:** algoritmen · **Varierar:** önskelistorna · **Urskiljs:** att minsnittet förklarar varför det inte går

## Vad är $m$ ? Linjär i vad, igen

Varje varv i Ford–Fulkerson är en grafsökning:  $O(|V| + |E|)$ , linjärt i flödesgrafens storlek. Flödet ökar med 1 per varv och är högst  $n$ , så det blir högst  $n$  varv.

| Om $m$ betyder ...                 | kanter i flödesgraf               | tid, $n$ varv |
|------------------------------------|-----------------------------------|---------------|
| högst $m$ saker per lista (Viggo)  | $O(nm)$                           | $O(n^2m)$     |
| totala antalet rader i alla listor | $O(n + m)$                        | $O(n(n + m))$ |
| antalet olika saker som finns      | $O(nm)$ (högst $n$ listor à $m$ ) | $O(n^2m)$     |

**⚠ Missuppfattning:** " $O(nm)$ , för grafen har  $O(nm)$  kanter." Det är ett varv. Det blir upp till  $n$  varv.

**Separation** Fast: algoritmen · **Varierar:** vad  $m$  står för · **Urskiljs:** att en  $O$ -formel bara betyder något när variablerna är definierade

## Vad betyder kapaciteterna? En i taget

| Ändring                                      | Betyder                                                | Rätt modell?             |
|----------------------------------------------|--------------------------------------------------------|--------------------------|
| $s \rightarrow \text{barn}: 2$               | varje barn kan få två klappar                          | ✓ (för en annan uppgift) |
| $\text{sak} \rightarrow t: k$                | det finns $k$ exemplar av saken                        | ✓                        |
| $\text{sak} \rightarrow t: \infty$           | obegränsat med exemplar: alla får RTX-kortet           | ✗ (Nvidia jublar)        |
| $s \rightarrow \text{barn}: \infty$          | ett barn kan få allt på sin lista                      | ✗                        |
| $\text{barn} \rightarrow \text{sak}: \infty$ | ingen skillnad: 1:orna vid $s$ och $t$ begränsar redan | ✓                        |

**⚠ Missuppfattning:** "Alla kapaciteter måste vara 1." Kanterna i mitten får vara vad som helst  $\geq 1$ . Det är kanterna vid  $s$  och  $t$  som uttrycker villkoren.

**Separation** Fast: grafen · **Varierar:** en kapacitet i taget · **Urskiljs:** vilken kapacitet som uttrycker vilket villkor

## Kaninhål: Samma sak utan flöde: alternerande stigar

Den sista förbättrande stigen, utan  $s$  och  $t$ :

|                   |      |        |
|-------------------|------|--------|
| Marcus – RTX      | ny   | +1     |
| RTX – Emma        | bort | -1     |
| Emma – PS5        | ny   | +1     |
| PS5 – Lovisa      | bort | -1     |
| Lovisa – Switch 2 | ny   | +1     |
| <hr/>             |      |        |
|                   |      | +1 par |

Börjar och slutar i omatchade hörn, växlar ny/bort.

Det är en förbättrande stig i flödesgrafen, sedd utan  $s$  och  $t$ .

**Berge (1957):** en matchning är störst  $\iff$  det inte finns någon alternerande stig mellan två omatchade hörn.

**Hopcroft–Karp (1973):** leta upp många kortaste alternerande stigar, disjunkta, i samma fas.  $O(\sqrt{|V|})$  faser:  $O(|E|\sqrt{|V|})$  i stället för  $O(|V||E|)$ .

## Vanliga fel i uppgift 4

**⚠ Missuppfattning:** "Girigt räcker." Girigt ger en *maximal* matchning. Här tre par, när fyra går.

**⚠ Missuppfattning:** "Flödet kan ge Emma och Marcus ett halvt grafikkort var." Heltalskapaciteter ger ett heltalsflöde. Ford–Fulkerson halverar inga grafikkort.

**⚠ Missuppfattning:** "Tiden är  $O(nm)$ ." Ett varv är  $O(nm)$ . Det blir upp till  $n$  varv:  $O(n^2m)$ .

**⚠ Missuppfattning:** "Kapacitet  $\infty$  på sak  $\rightarrow t$ , för säkerhets skull." Då får alla samma sak.

**Kontrast** **Fast:** uppgiften · **Varierar:** modell och analys, rätt och nästan rätt · **Urskiljs:** girig/största, heltal, varv, kapaciteter

## 5

## Undre gräns för sökning i sorterad array

**Lärandeobjekt:** bevisa en undre gräns för ett *problem* — och hålla isär den från gränser för algoritmer och från bästa och värsta fall.

**Kritiska aspekter** (det vi kommer att variera):

- ▶ Gränsens riktning ( $O$ ,  $\Omega$ ,  $\Theta$ ) och fallet (bästa, medel, värsta) är två olika saker.
- ▶ Algoritm eller problem:  $\exists A \forall x$  mot  $\forall A \exists x$ .
- ▶ Beslutsträdet: löven är möjliga svar, höjden är värsta fallet.

# Övre och undre gräns är inte bästa och värsta fall

|                      | bästa fallet<br>$T_{\min}(n)$ | medelfallet<br>$T_{\text{medel}}(n)$ | värsta fallet<br>$T_{\max}(n)$ |
|----------------------|-------------------------------|--------------------------------------|--------------------------------|
| $O$ övre gräns       |                               |                                      | $O = \text{värsta}$            |
| $\Omega$ undre gräns | $\Omega = \text{bästa}$       | <b>Nej.</b>                          |                                |
| $\Theta$ båda        |                               | $\Theta = \text{medel}$              |                                |

En vanlig bild: tre synonymer.  $O$  betyder värsta fallet,  $\Omega$  bästa,  $\Theta$  medel. Den är fel, och den gör resten av kursen svårare än den behöver vara.

## Övre och undre gräns är inte bästa och värsta fall

|                      | bästa fallet<br>$T_{\min}(n)$ | medelfallet<br>$T_{\text{medel}}(n)$ | värsta fallet<br>$T_{\max}(n)$ |
|----------------------|-------------------------------|--------------------------------------|--------------------------------|
| $O$ övre gräns       | $O(n)$                        | $O(n^2)$                             | $O(n^2)$                       |
| $\Omega$ undre gräns | $\Omega(n)$                   | $\Omega(n^2)$                        | $\Omega(n^2)$                  |
| $\Theta$ båda        | $\Theta(n)$                   | $\Theta(n^2)$                        | $\Theta(n^2)$                  |

▲ Bästa fallet har ett ord

▲ Värsta fallet har ett  $\Omega$

Insättningssortering, alla nio rutor. **Fallet** väljer *vilken funktion* vi pratar om.  $O$ ,  $\Omega$ ,  $\Theta$  säger hur *den* växer: uppåt begränsad, nedåt begränsad, eller båda.

# Övre och undre gräns är inte bästa och värsta fall

|                      | bästa fallet<br>$T_{\min}(n)$    | medelfallet<br>$T_{\text{medel}}(n)$ | värsta fallet<br>$T_{\max}(n)$            |
|----------------------|----------------------------------|--------------------------------------|-------------------------------------------|
| $O$ övre gräns       | $O(n)$<br>också $O(n^2), O(n^3)$ | $O(n^2)$<br>också $O(n^3)$           | $O(n^2)$<br>också $O(n^3), O(2^n)$        |
| $\Omega$ undre gräns | $\Omega(n)$<br>också $\Omega(1)$ | $\Omega(n^2)$<br>också $\Omega(n)$   | $\Omega(n^2)$<br>också $\Omega(n \log n)$ |
| $\Theta$ båda        | $\Theta(n)$<br>bara den klassen  | $\Theta(n^2)$<br>bara den klassen    | $\Theta(n^2)$<br>bara den klassen         |

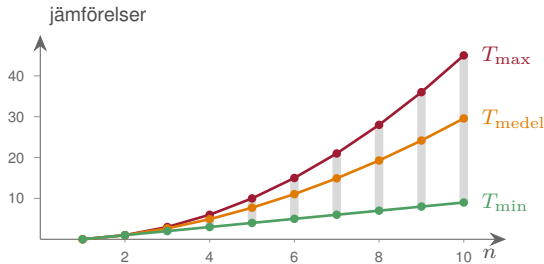
▲ Bästa fallet har ett ord

▲ Värsta fallet har ett  $\Omega$

$O$  och  $\Omega$  får vara lösa och fortfarande sanna.  $\Theta$  får inte det. "Värsta fallet är  $O(2^n)$ " är sant — och helt ointressant.

# Tre funktioner, inte en

## Insättningssortering: jämförelser



För varje  $n$  finns  $n!$  indata (grå staplar). Varje fall plockar ut en funktion av  $n$ :

- ▶  $T_{\min}(n) = n - 1$ : snabbaste indata (redan sorterat).
- ▶  $T_{\max}(n) = \binom{n}{2}$ : långsammaste (omvänt sorterat).
- ▶  $T_{\text{medel}}(n) \approx n^2/4$ : genomsnittet över alla ordningar.

Sedan beskriver  $O$ ,  $\Omega$  och  $\Theta$  den funktionen.

"Värsta fallet är  $\Omega(n^2)$ " och "bästa fallet är  $O(n)$ " är helt vanliga, sanna meningar.

**Separation** Fast: algoritmen · Varierar: fallet · Urskiljs: att fallet väljer funktionen och  $O/\Omega$  begränsar den

## En tredje axel: algoritm eller problem?

|                                | Övre gräns                                                                                                                 | Undre gräns                                                                                                                   |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>Algoritm <math>A</math></b> | $A$ tar högst $f(n)$ på <i>varje</i> indata av storlek $n$ .<br>Binärsökning: $O(\log n)$ .                                | <i>Något</i> indata får $A$ att ta minst $g(n)$ .<br>Linjärsökning: $\Omega(n)$ i värsta fallet.                              |
| <b>Problem <math>P</math></b>  | <i>Det finns</i> en algoritm som klarar $f(n)$ : $\exists A \forall x$ .<br>Sökning: $O(\log n)$ , tack vare binärsökning. | <i>Varje</i> algoritm har ett indata som kostar $g(n)$ :<br>$\forall A \exists x$ .<br>Sökning: $\Omega(\log n)$ — uppgift 5. |

Möts gränserna är problemet  $\Theta(\log n)$  och binärsökning optimal. En undre gräns för problemet måste gälla även algoritmer som ingen har skrivit än.

**⚠ Missuppfattning:** "Linjärsökning tar  $n$  steg, alltså är sökning  $\Omega(n)$ ." En gräns för en dålig algoritm, inte för problemet.

**Separation** Fast: sökning i sorterad array · **Varierar:** algoritm eller problem · **Urskiljs:** kvantorerna  $\exists A$  och  $\forall A$

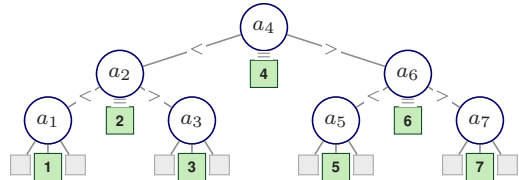
## Uppgift 5: Undre gräns för sökning i sorterad array

Binärsökning i en sorterad array med  $n$  tal tar som bekant tiden  $O(\log n)$ . Bevisa att  $\Omega(\log n)$  också är en undre gräns för antalet jämförelser som krävs för detta problem (i värsta fallet).

**Modell:** algoritmen får bara jämföra  $x$  med arrayelement:  $x < a_i$ ,  $x = a_i$  eller  $x > a_i$ .  
Varje jämförelse kostar 1, allt annat är gratis.

**Beslutsträd:** en inre nod per jämförelse, ett löv per svar. En körning är en stig från roten till ett löv. Värsta fallet är trädets höjd.

**Binärsökningens träd,  $n = 7$**



gröna löv: hittad på plats  $i$     grå löv: finns inte

## Beviset: räkna löven

1. Möjliga svar: " $x = a_i$ " för  $i = 1, \dots, n$ , eller "finns inte". Minst  $n + 1$  olika svar, alltså minst  $n + 1$  löv.
2. Varje inre nod har högst 3 barn. Ett träd med höjd  $h$  har högst  $3^h$  löv.
3.  $3^h \geq n + 1 \Rightarrow h \geq \log_3(n + 1) = \Omega(\log n)$ .

Det gäller *varje* beslutsträd, alltså varje jämförelsebaserad algoritm. Ingen balans behöver antas.

**Skarpare:** svaret " $x = a_i$ " kräver en nod som jämför med  $a_i$  och får  $=$ . Alltså minst  $n$  inre noder. Utan  $=$ -löven är det ett binärträd med minst  $n$  noder, och dess höjd är minst  $\lceil \log_2(n + 1) \rceil$ .

Binärsökning gör exakt så många jämförelser. Den är optimal — inte bara asymptotiskt.

**Generalisering** Fast: argumentet: räkna svar, jämför med höjd · **Varierar:** algoritmen · **Urskiljs:** att gränsen gäller alla träd på en gång

## **Kaninhål: Samma bevis, berättat som en motståndare**

Motståndaren bestämmer inte arrayen i förväg. Den håller reda på vilka utfall som fortfarande är möjliga:  $n$  träffar och  $n + 1$  luckor,  $2n + 1$  stycken.

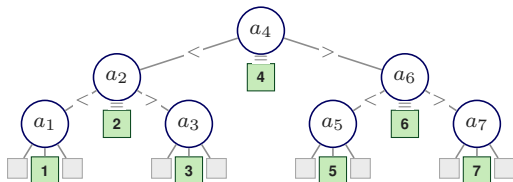
- ▶ Vid varje jämförelse  $x$  mot  $a_i$  svarar den  $<$  eller  $>$ , åt det håll där flest utfall finns kvar. Aldrig = så länge den kan undvika det.
- ▶ Från  $m$  möjliga utfall finns minst  $(m - 1)/2$  kvar efter frågan.
- ▶ Algoritmen kan inte svara förrän ett enda utfall återstår. Räkningen ger  $2^k \geq n + 1$ , alltså minst  $\lceil \log_2(n + 1) \rceil$  jämförelser.

Motståndaren *är* beslutsträdet: den väljer den längsta stigen genom algoritmens träd. Samma argument, två berättelser. Uppgift 6 använder den andra.

**Fusion Fast:** påståendet  $\Omega(\log n)$  · **Varierar:** bevistekniken: räkna löv eller spela motståndare · **Urskiljs:** att en motståndare är den värsta stigen i trädet

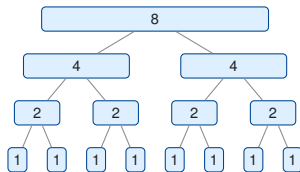
# Beslutsträd är inte rekursionsträd

## Beslutsträd (binärsökning)



En körning = *en stig*. Värsta fallet = höjden.

## Rekursionsträd (mergesort, $n = 8$ )



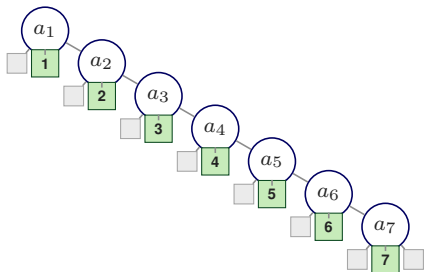
En körning = *hela trädet*. Tid = summan.

**⚠ Missuppfattning:** "Binärsökningens rekursion har djup  $\log n$ , alltså är  $\Omega(\log n)$  bevisat." Det säger vad *binärsökning* gör, inte vad alla algoritmer måste göra.

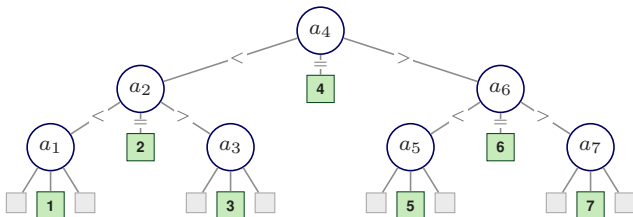
**Kontrast Fast:** "träd som beskriver en algoritm" · **Varierar:** vad en körning är: stig eller hela trädet · **Urskiljs:** att beslutsträdet beskriver alla körningar

## Två algoritmer, två träd, lika många löv

Linjärsökning: höjd 7



Binärsökning: höjd 3

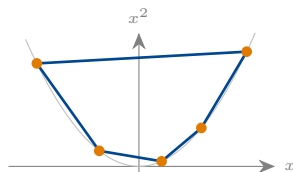


Båda har 7 gröna och 8 grå löv. Gränsen säger inte att alla träd är balanserade. Den säger att *ingen* träd med så många löv kan vara lägre än  $\lceil \log_2 8 \rceil = 3$ .

**Generalisering** Fast: problemet och antalet löv · Varierar: algoritmen, alltså trädets form · Urskiljs: att den undre gränsen gäller varje form

# Fyra verktyg för undre gränser

| Verktyg         | Idé                                               | Idag                                  |
|-----------------|---------------------------------------------------|---------------------------------------|
| Läsa och skriva | Svaret har storlek $s \Rightarrow \Omega(s)$      | Euler: $\Omega( E )$                  |
| Beslutsträd     | $L$ möjliga svar $\Rightarrow$ höjd $\geq \log L$ | Uppgift 5; sortering                  |
| Motståndare     | Svara elakt men konsekvent                        | Uppgift 5 igen; uppgift 6             |
| Reduktion       | Löser du $B$ snabbt, löser du $A$ snabbt          | Sortering $\rightarrow$ konvext hölje |



Sortera  $x_1, \dots, x_n$ : beräkna konvexa höljet av punkterna  $(x_i, x_i^2)$ . Höljet går igenom dem i sorterad ordning.

Reduktionen går *från* det kända problemet *till* det nya: om konvexa höljet gick snabbare än  $\Omega(n \log n)$  skulle sortering också göra det.

## Kaninhål: Sortering: $n!$ löv

- ▶ En jämförelsebaserad sortering måste kunna ge var och en av de  $n!$  ordningarna som svar.
- ▶ Jämförelsen  $a_i < a_j$  har två utfall. Ett binärt träd med höjd  $h$  har högst  $2^h$  löv.
- ▶  $2^h \geq n! \Rightarrow h \geq \log_2 n! \geq \frac{n}{2} \log_2 \frac{n}{2} = \Omega(n \log n)$ , eftersom  $n!$  har minst  $n/2$  faktorer som är  $\geq n/2$ .
- ▶ Mergesort klarar  $O(n \log n)$ . Problemet är alltså  $\Theta(n \log n)$ .

Samma recept som i uppgift 5, fler löv. För  $n = 7$ :  $\lceil \log_2 5040 \rceil = 13$  jämförelser krävs i värsta fallet. Det finns algoritmer som klarar sig med exakt 13, men ingen känd formel för det bästa antalet i allmänhet.

## **Kaninhål: Modellen spelar roll**

Undre gränsen gäller algoritmer som bara *jämför*. Den som räknar med nycklarna har inte lovat något:

- ▶ **Interpolationssökning** gissar var  $x$  borde ligga utifrån värdet. Jämnt fördelade nycklar:  $O(\log \log n)$  i medel. Värsta fallet är fortfarande  $\Theta(n)$ .
- ▶ **Hashning**:  $O(1)$  förväntat. Behöver inte ens sorterad ordning.
- ▶ **Räknesortering, radixsortering**:  $O(n + k)$ . "Sortering är  $\Omega(n \log n)$ " gäller jämförelser.

**⚠ Missuppfattning:** "Hashning motbevisar uppgift 5." Nej: uppgift 5 handlar om jämförelser. En undre gräns är alltid en gräns *i en modell*.

**Kontrast** **Fast:** problemet: hitta  $x$  · **Varierar:** beräkningsmodellen · **Urskiljs:** att en undre gräns bara gäller i sin modell

## Vanliga fel i uppgift 5

⚠ **Missuppfattning:** "Trädet har  $n$  löv." Svaret "finns inte" är också ett svar: minst  $n + 1$ .

⚠ **Missuppfattning:** "Höjden är  $O(\log n)$ ." Det gäller binärsökningens träd. Beviset behöver att *varje* träd har höjd  $\Omega(\log n)$ .

⚠ **Missuppfattning:** "Trädet är balanserat, så höjden är  $\log n$ ." Balans får man inte anta: trädet tillhör en okänd algoritm. Ett träd med  $L$  löv har höjd  $\geq \log L$  ändå.

⚠ **Missuppfattning:** "Jag hittade ett indata där min algoritm är långsam." Det är en undre gräns för *din* algoritm. Problemet kräver  $\forall A$ .

**Kontrast** Fast: uppgiften · Varierar: beviset, rätt och nästan rätt · Urskiljs: löv, riktning, balans, kvantor

## 6

## Laga julgransbelysning

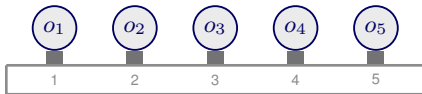
**Lärandeobjekt:** analysera en algoritm *exakt* och bevisa en matchande undre gräns för problemet.

**Kritiska aspekter** (det vi kommer att variera):

- ▶ Kostnadsmåttet: lampbyten, inte tester och inte tid.
- ▶ Algoritmens värsta fall mot problemets undre gräns.
- ▶ Exakt antal,  $2n - 1$ , inte bara  $\Theta(n)$ .

## Uppgift 6: Laga julgransbelysning

Jonas julgransbelysning med  $n$  seriellt kopplade lampor lyser inte. Det räcker att en lampa är trasig för att belysningen inte ska lysa. Han har  $n$  nya lampor som säkert fungerar, men vill inte ersätta några hela lampor med nya. En urskruvning följt av en iskruvning tar tiden 1. Konstruera en algoritm, analysera värstafallet *exakt* och ge en matchande undre gräns.



- **Kostar 1:** ett byte (ur + i).
- **Gratis:** att titta om det lyser.
- **Förbjudet:** att ersätta en hel gammal lampa.

**Separation** Fast: problemet · **Varierar:** kostnads måttet · **Urskiljs:** att vi räknar byten, inte tester

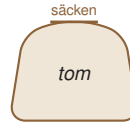
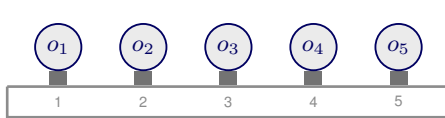
# Viggos algoritm

- 1: byt alla lampor utom den på plats 1 mot nya
- 2: lägg de gamla lamporna i en säck
- 3:  $i \leftarrow 1$
- 4: **medan** det finns lampor i säcken **gör**
- 5:     **om** belysningen lyser **så**
- 6:          $i \leftarrow i + 1$
- 7:         skruva ur den nya lampan på plats  $i$
- 8:     **annars**
- 9:         skruva ur den gamla trasiga lampan på plats  $i$
- 10:     ta en lampa ur säcken och skruva i den på plats  $i$
- 11: **om** belysningen inte lyser **så**
- 12:     byt lampan på plats  $i$  mot en ny

**Invariant** (i början av varje varv): platserna före  $i$  har gamla, hela lampor; plats  $i$  har en gammal lampa; platserna efter  $i$  har nya.

Idén: när alla andra lampor är kända och hela avslöjar belysningen exakt en lampa. Alla gamla lampor provas en i taget, på samma sätt.

## Viggos algoritm steg för steg ( $n = 5$ , trasiga: $o_2$ och $o_5$ )



trasiga: —

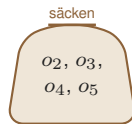
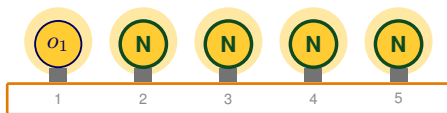
**byten: 0**

Det lyser inte. Minst en lampa är trasig — men vilka?

Invariant: platserna före  $i$  har hela gamla lampor, plats  $i$  en gammal lampa, platserna efter  $i$  nya. (Utropen är lånade från Alice.)

**Generalisering** Fast: algoritmen · **Varierar:** vilka lampor som är trasiga · **Urskiljs:** att varje gammal lampa provas en gång

## Viggos algoritm steg för steg ( $n = 5$ , trasiga: $o_2$ och $o_5$ )



trasiga: —

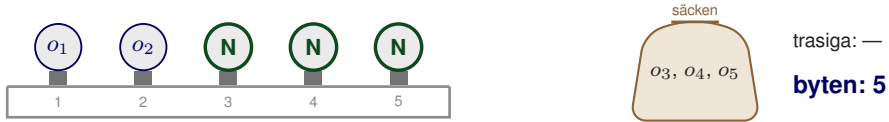
**byten: 4**

Byt alla utom plats 1 mot nya: 4 byten. De gamla i säcken. Det lyser! *Fantastiskt*.  $o_1$  är hel.

Invariant: platserna före  $i$  har hela gamla lampor, plats  $i$  en gammal lampa, platserna efter  $i$  nya. (Utropen är lånade från Alice.)

**Generalisering** **Fast:** algoritmen · **Varierar:** vilka lampor som är trasiga · **Urskiljs:** att varje gammal lampa provas en gång

## Viggos algoritm steg för steg ( $n = 5$ , trasiga: $o_2$ och $o_5$ )



Plats 2: ny ut,  $o_2$  in. Mörkt. *Ånej.*  $o_2$  är trasig.

Invariant: platserna före  $i$  har hela gamla lampor, plats  $i$  en gammal lampa, platserna efter  $i$  nya. (Utropen är lånade från Alice.)

**Generalisering** Fast: algoritmen · **Varierar:** vilka lampor som är trasiga · **Urskiljs:** att varje gammal lampa provas en gång

## Viggos algoritm steg för steg ( $n = 5$ , trasiga: $o_2$ och $o_5$ )



trasiga:  $o_2$

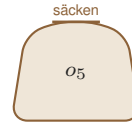
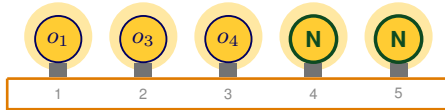
**byten: 6**

$o_2$  ut,  $o_3$  in på samma plats. Det lyser. *Otroligt.*  $o_3$  är hel.

Invariant: platserna före  $i$  har hela gamla lampor, plats  $i$  en gammal lampa, platserna efter  $i$  nya. (Utropen är lånade från Alice.)

**Generalisering** Fast: algoritmen · **Varierar:** vilka lampor som är trasiga · **Urskiljs:** att varje gammal lampa provas en gång

## Viggos algoritm steg för steg ( $n = 5$ , trasiga: $o_2$ och $o_5$ )



trasiga:  $o_2$

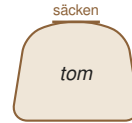
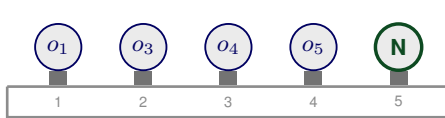
**byten: 7**

Plats 3: ny ut,  $o_4$  in. Det lyser. *Häpnadsväckande.*

Invariant: platserna före  $i$  har hela gamla lampor, plats  $i$  en gammal lampa, platserna efter  $i$  nya. (Utropen är lånade från Alice.)

**Generalisering** Fast: algoritmen · **Varierar:** vilka lampor som är trasiga · **Urskiljs:** att varje gammal lampa provas en gång

## Viggos algoritm steg för steg ( $n = 5$ , trasiga: $o_2$ och $o_5$ )



trasiga:  $o_2$

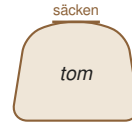
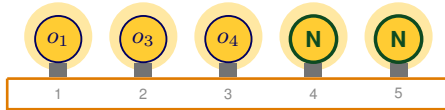
**byten: 8**

Plats 4: ny ut,  $o_5$  in. Mörkt:  $o_5$  är trasig. Säcken är tom.

Invariant: platserna före  $i$  har hela gamla lampor, plats  $i$  en gammal lampa, platserna efter  $i$  nya. (Utropen är lånade från Alice.)

**Generalisering** Fast: algoritmen · **Varierar:** vilka lampor som är trasiga · **Urskiljs:** att varje gammal lampa provas en gång

## Viggos algoritm steg för steg ( $n = 5$ , trasiga: $o_2$ och $o_5$ )



trasiga:  $o_2, o_5$

**byten: 9**

$o_5$  ut, en ny in. *Hilvide*.  $9 = 2n - 1$  byten.

Invariant: platserna före  $i$  har hela gamla lampor, plats  $i$  en gammal lampa, platserna efter  $i$  nya. (Utropen är lånade från Alice.)

**Generalisering** Fast: algoritmen · **Varierar:** vilka lampor som är trasiga · **Urskiljs:** att varje gammal lampa provas en gång

## Exakt analys: det är sista lampan som avgör

$$\underbrace{(n-1)}_{\text{byt till nya}} + \underbrace{(n-1)}_{\text{prova säcken}} + \underbrace{[o_n \text{ trasig}]}_{\text{sista bytet}}$$

Säcken ger alltid  $n - 1$  byten. Sista bytet behövs bara om den sista lampan ur säcken är trasig.

- Värsta fallet:  $2n - 1$ .
- Bästa fallet:  $2n - 2$ .

| Trasiga ( $n = 5$ )   | Byten    |
|-----------------------|----------|
| $o_2, o_5$ (exemplet) | 9        |
| bara $o_5$            | <b>9</b> |
| alla fem              | 9        |
| bara $o_1$            | 8        |
| bara $o_2$            | 8        |
| $o_1, \dots, o_4$     | 8        |

**⚠ Missuppfattning:** "Värsta fallet är när alla lampor är trasiga." Det är *ett* värsta fall. Det räcker att den sista lampan i säcken är trasig.

**Separation** Fast: algoritmen · **Varierar:** vilka lampor som är trasiga · **Urskiljs:** att bara  $o_n$  påverkar kostnaden

## **Kaninhål: Mörkret i början är gratis information**

Belysningen lyste inte från början. Alltså är minst en lampa trasig.

- ▶ Har  $o_1, \dots, o_{n-1}$  alla visat sig hela vet vi att  $o_n$  är trasig — utan att prova. Låt den nya lampan sitta kvar.
- ▶ På just det indatat kostar det  $2n - 3$  i stället för  $2n - 1$ .
- ▶ Värsta fallet blir inte bättre: med  $o_{n-1}$  och  $o_n$  trasiga kostar det fortfarande  $2n - 1$ .

Det kan det inte heller bli. Att optimera bort det ena värsta fallet flyttar det bara till ett annat. Det är vad en undre gräns betyder i praktiken.

**Kontrast** Fast: problemet · **Varierar**: algoritmen: med eller utan knepet · **Urskiljs**: att bästa fallet kan förbättras, värsta inte

## Undre gräns: motståndaren

Påstående: *varje* algoritm behöver  $2n - 1$  byten i värsta fallet. Viggos motståndare:

1. En lampa avslöjas bara när alla *andra* lampor i belysningen är kända och hela.
2. Motståndaren låter belysningen vara mörk tills  $n - 1$  nya lampor sitter i: den sista originallampan är trasig. ( $n - 1$  byten.)
3. De  $n - 1$  bortskruvade lamporna har inte avslöjat något. Var och en måste in igen. ( $n - 1$  byten till.)
4. Den sist provade är också trasig. (1 byte.)

Summa  $2n - 1$ , med bara två trasiga lampor.

**⚠ Missuppfattning:** "Undre gräns för algoritmen" (rubrik i en äldre bild). Argumentet gäller varje algoritm: det är en undre gräns för *problemet*. För bara Viggos algoritm räcker ett indata:  $O_n$  trasig.

## **Kaninhål: Ett stramare bevis: två indata som ser likadana ut**

Kör en godtycklig algoritm på  $X$ : *alla* lampor trasiga. Jämför med  $X_j$ : bara  $o_j$  hel.

- ▶ Observationerna skiljer sig bara i läget  $S_j$ :  $o_j$  sitter i och alla andra platser har nya lampor. (Då lyser  $X_j$ , men inte  $X$ .)
- ▶ Slutlägena måste skilja sig:  $o_j$  ska sitta kvar i  $X_j$  men inte i  $X$ . Körningen på  $X$  måste alltså passera *alla*  $S_1, \dots, S_n$ .
- ▶ Minst  $n - 1$  byten till det första  $S_j$ . Minst 1 byte mellan två olika  $S_j$ :  $n - 1$  gånger. Minst 1 byte till slutläget.

$$(n - 1) + (n - 1) + 1 = 2n - 1 \quad \text{byten för varje algoritm, på indatat } X.$$

Viggos algoritm går  $S_1 \rightarrow S_2 \rightarrow \dots \rightarrow S_n$  med ett byte i taget på  $X$ . Den kan inte göras bättre.

## Samma idé i uppgift 5 och 6

|                     | Uppgift 5: sökning            | Uppgift 6: lampor            |
|---------------------|-------------------------------|------------------------------|
| Kostnad             | jämförelser                   | byten (testerna är gratis)   |
| Observation         | $<, =$ eller $>$              | lyser eller lyser inte       |
| Motståndaren håller | den större halvan av utfallen | mörkret så länge som möjligt |
| Undre gräns         | $\lceil \log_2(n + 1) \rceil$ | $2n - 1$                     |
| Når gränsen         | binärsökning                  | Viggos algoritim             |

En motståndare väljer den dyraste stigen genom algoritmens beslutsträd.

**⚠ Missuppfattning:** "Binärsök efter den trasiga lampan:  $O(\log n)$ ." Det sparar *tester*, som är gratis. Bytena kostar: minst  $2n - 1$ , oavsett metod.

**Fusion Fast:** bevisidén · **Varierar:** problem, kostnadsmått och observationer samtidigt · **Urskiljs:** motståndare = värsta stigen

## Vanliga fel i uppgift 6

⚠ **Missuppfattning:** " $\Theta(n)$ ." Uppgiften vill ha exakt antal:  $2n - 1$ . Ordoklass räcker inte när skillnaden mellan  $2n - 1$  och  $2n - 2$  är poängen.

⚠ **Missuppfattning:** "Byt alla mot nya:  $n$  byten." Hela gamla lampor får inte ersättas.

⚠ **Missuppfattning:** "Min algoritm tar  $2n - 1$ , alltså är det en undre gräns." Det är algoritmens värsta fall. Problemets undre gräns kräver ett argument för alla algoritmer.

⚠ **Missuppfattning:** "Värsta fallet: alla trasiga." Ett av flera. För Viggos algoritm räcker  $o_n$ .

**Kontrast** **Fast:** uppgiften · **Varierar:** analysen, rätt och nästan rätt · **Urskiljs:** exakt, tillåtet, algoritm eller problem

## Det vi varierade

| Uppgift           | Kritisk aspekt                       | Det man ska kunna se                                                                 |
|-------------------|--------------------------------------|--------------------------------------------------------------------------------------|
| 1 Spännande träd  | kandidatmängden                      | Krusk <b>AL</b> är glob <b>AL</b> ; Prim tittar bara ut ur trädet                    |
| 2 Förändrat flöde | kanten och minsnitten                | +1 hjälper om kanten är i <i>alla</i> minsnitt, −1 stjälper om den är i <i>något</i> |
| 3 Eulercykel      | datastrukturen                       | linjärt i $ V  +  E $ kräver delade kanter och pekare per hörn                       |
| 4 Julklappar      | modellen och $m$                     | girigt ger maximal, flöde ger största; $O(\dots)$ kräver definierade variabler       |
| 5 Sökning         | fall mot gräns, algoritm mot problem | bästa fallet har ett ordo; undre gränser gäller alla algoritmer                      |
| 6 Lampor          | kostnadsmåttet                       | exakt $2n - 1$ , för varje algoritm                                                  |

Den återkommande frågan: **linjär i vad?**  $|V| + |E|$ ,  $|V|^2$ ,  $n$ , antal byten — eller ett tal i indata, och då är det inte linjärt alls.



# Frågor?

(Undre gräns: 0.)